



HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



SOICT

School of Information and Communication Technology

ONE LOVE. ONE FUTURE.



TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Natural Language Processing

2 – Text Embeddings – rNN – LSTM – Seq2Seq – Attention

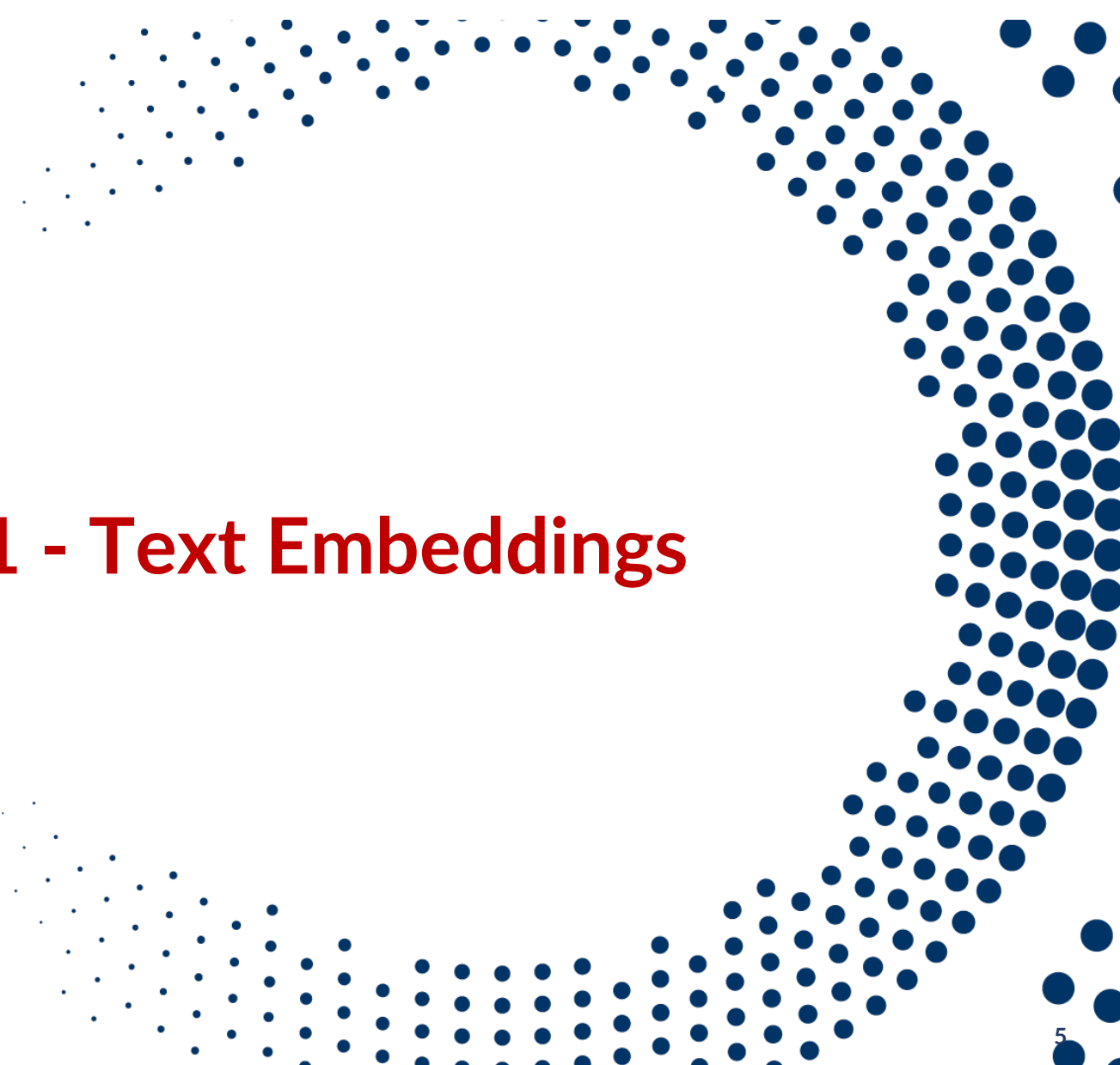
ONE LOVE. ONE FUTURE.

FYI

- *Lecturer: Dr. Bui Thi Mai Anh*
- **Research domains:**
 - **Software Engineering**
 - **Applied AI in Software Engineering**
 - **Search-based Software Engineering:**
evolutionary computing,
reinforcement learning, deep learning
 - **NLP4SE**
- **Lab: Intelligent Software Engineering – BK-AI Center**

<https://iselab-dorlab.github.io/>





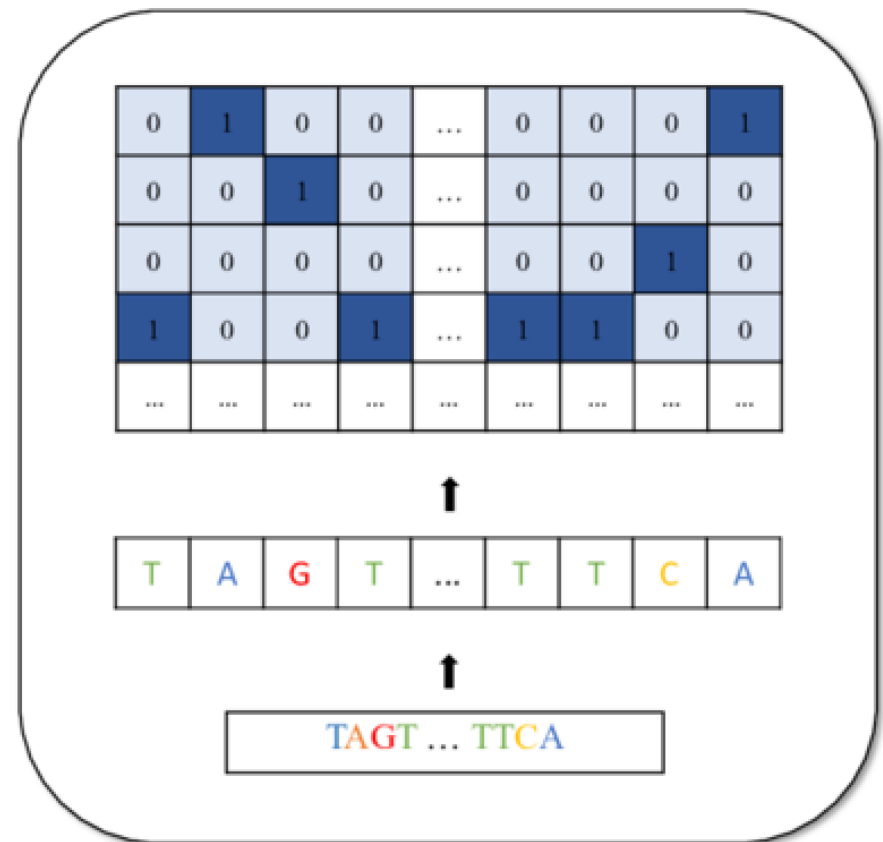
Section 1 - Text Embeddings

ONE LOVE. ONE FUTURE.

Discrete Representation

- **One hot encoding: one-to-one** correspondence between text units and representation element
- BOW: describes a document according to which words are present, ignoring word ordering
- TF-IDF: weighs words with TF & IDF

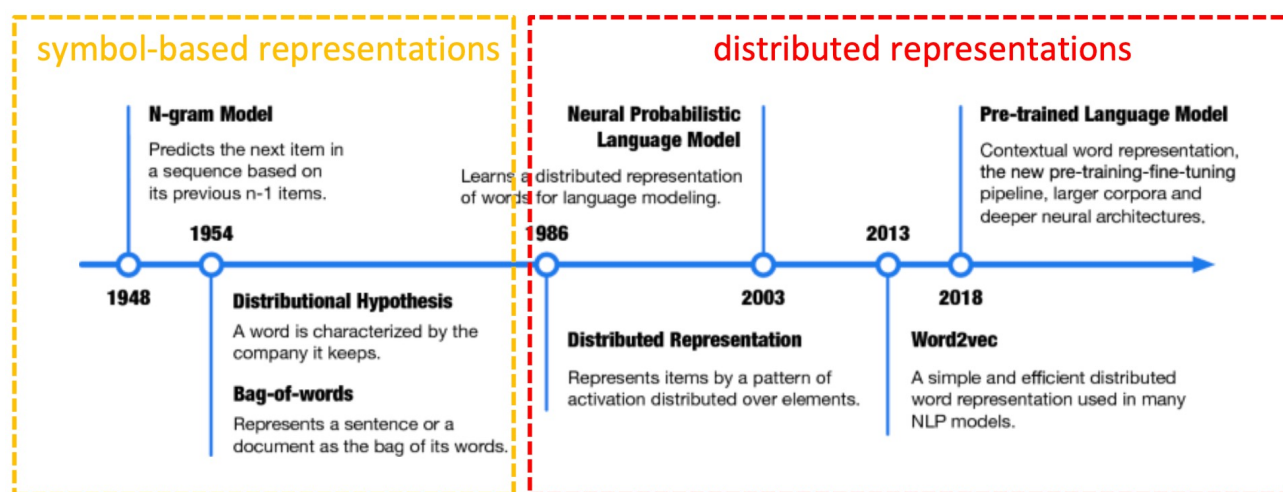
One-hot encoding



Issues: Many dimensions needed – vectors do not reflect semantic similarity

Distributed Text Representations

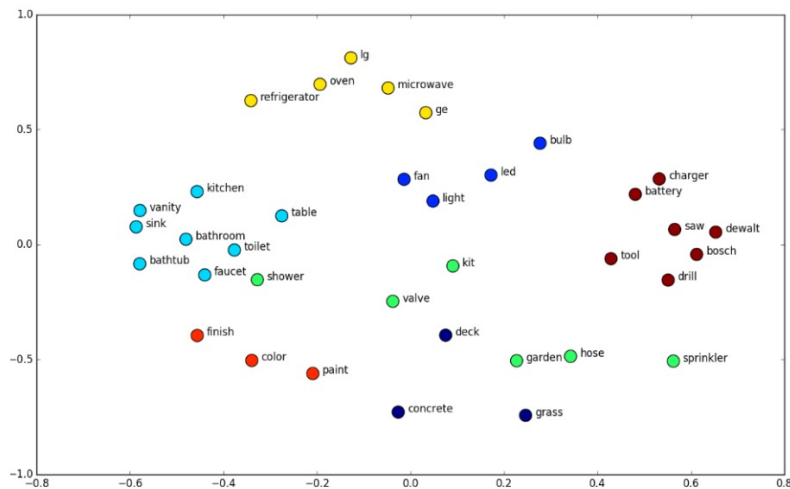
- Word that are used and occur in the **same contexts** tend to have **similar meanings**
- **Text Embeddings:** distributed representation
 - Representation of any text units is **distributed** over all vector dimensions as **continuous values**
 - Vectors are **dense, lower-dimensional**
- Representations are learned based on hypothesis: **vector space similarity = semantic similarity**



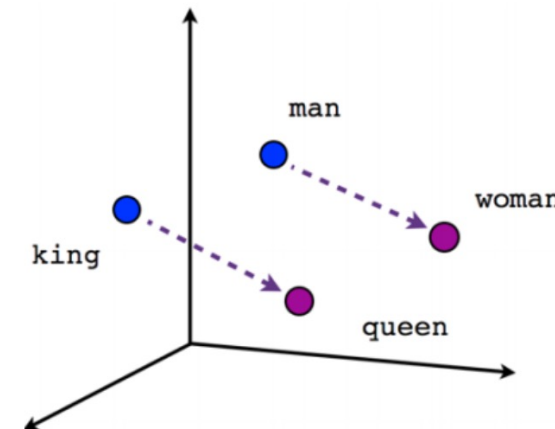
Better at capturing the word semantic!

Text Embeddings

- Embed one-hot vector into lower-dimensional space
- Word embedding captures useful properties of word semantics
 - Word similarity: Words with similar meanings are embedded closer
 - Word analogy: Linear relationship between words



Word Similarity

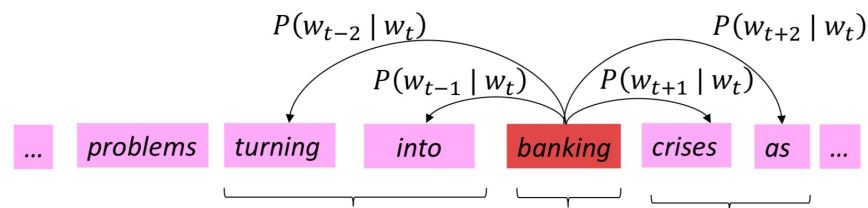


Word Analogy

Word2Vec

The main idea:

- Start with random vectors
- Iterate through each word position in the whole corpus
- Try to predict surrounding words using word vectors



- Learning: update vectors so they can predict actual surrounding words better
- The word embedding model try to learn word vectors that capture well word similarity and meaningful directions in a word space!

Word2Vec

The **output probabilities** are going to relate to how likely it is finding each vocabulary word nearby our **input word**

- Example: the input word “SoViet”, the output probabilities are going to be much higher for words like “Union” and “Russia” than for unrelated words like “watermelon” and “kangaroo”

$$P(\text{“Union”} | \text{“SoViet”}) > P(\text{“watermelon”} | \text{“SoViet”})$$



Training words on a simple neural network (with one hidden layer)

Source Text	Training Samples					
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡	The	quick	brown	(the, quick) (the, brown)		
The	quick	brown				
The <table><tr><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)		
quick	brown	fox				
The quick <table><tr><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)		
brown	fox	jumps				
The <table><tr><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡	quick	brown	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
quick	brown	fox	jumps	over		

Window size (context) is a hyperparameter

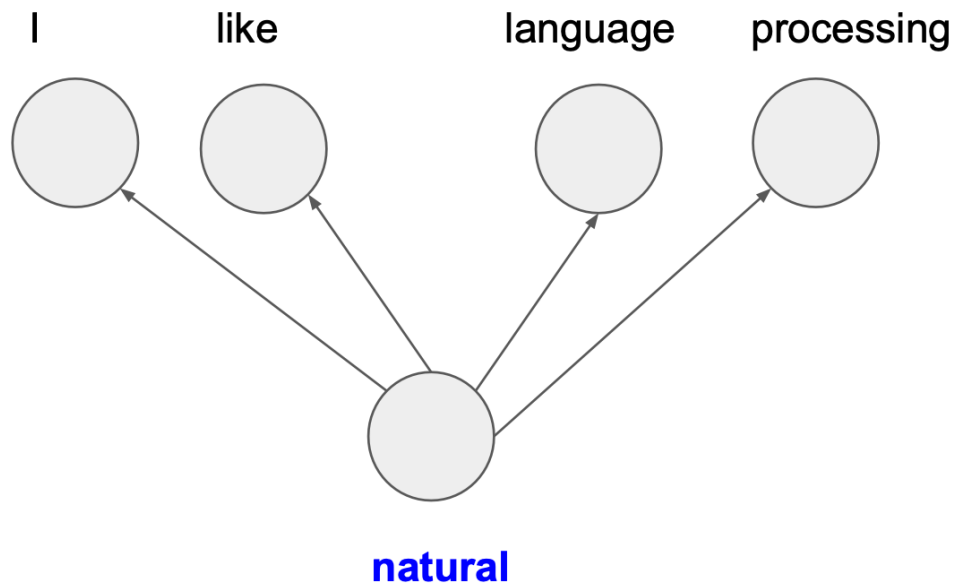
The network is going to learn the statistics from the number of times each pairing shows up



Word2Vec (Skip-Gram Model)

- Two assumptions:

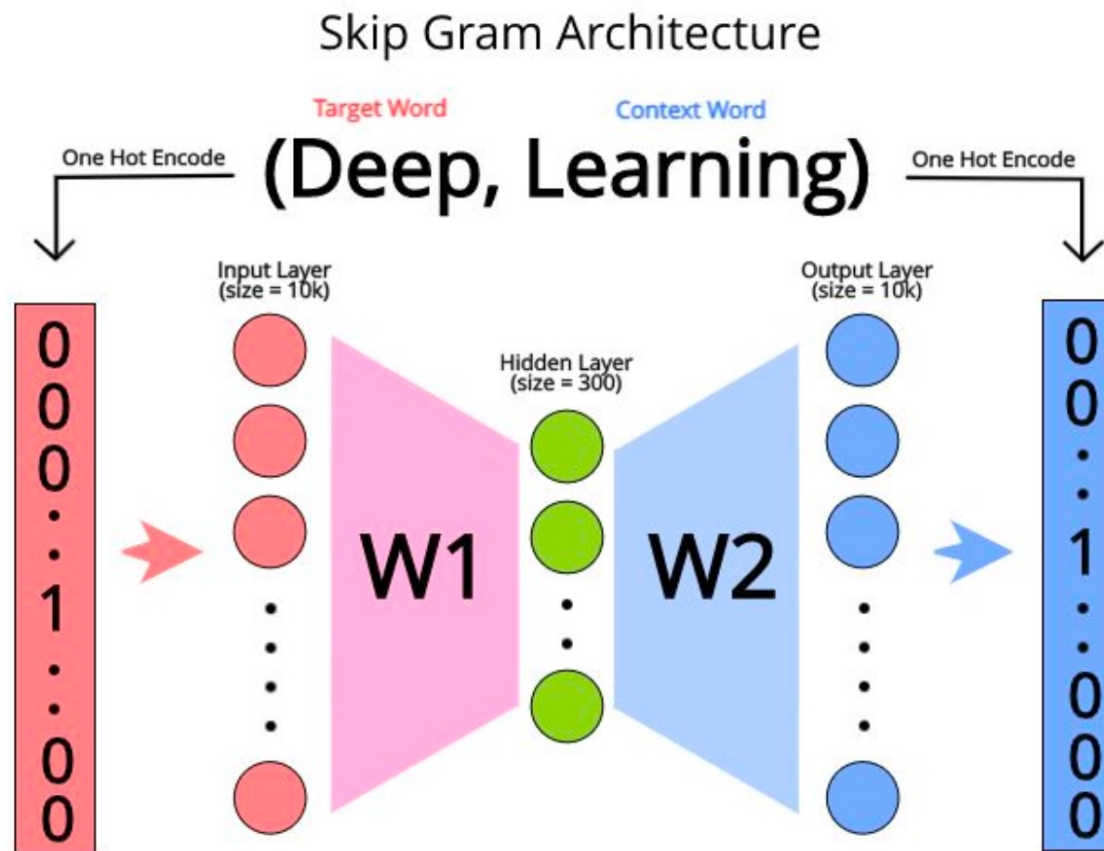
- A word can be used to generate the words surrounding it
- Given the center word, the context words are generated independently



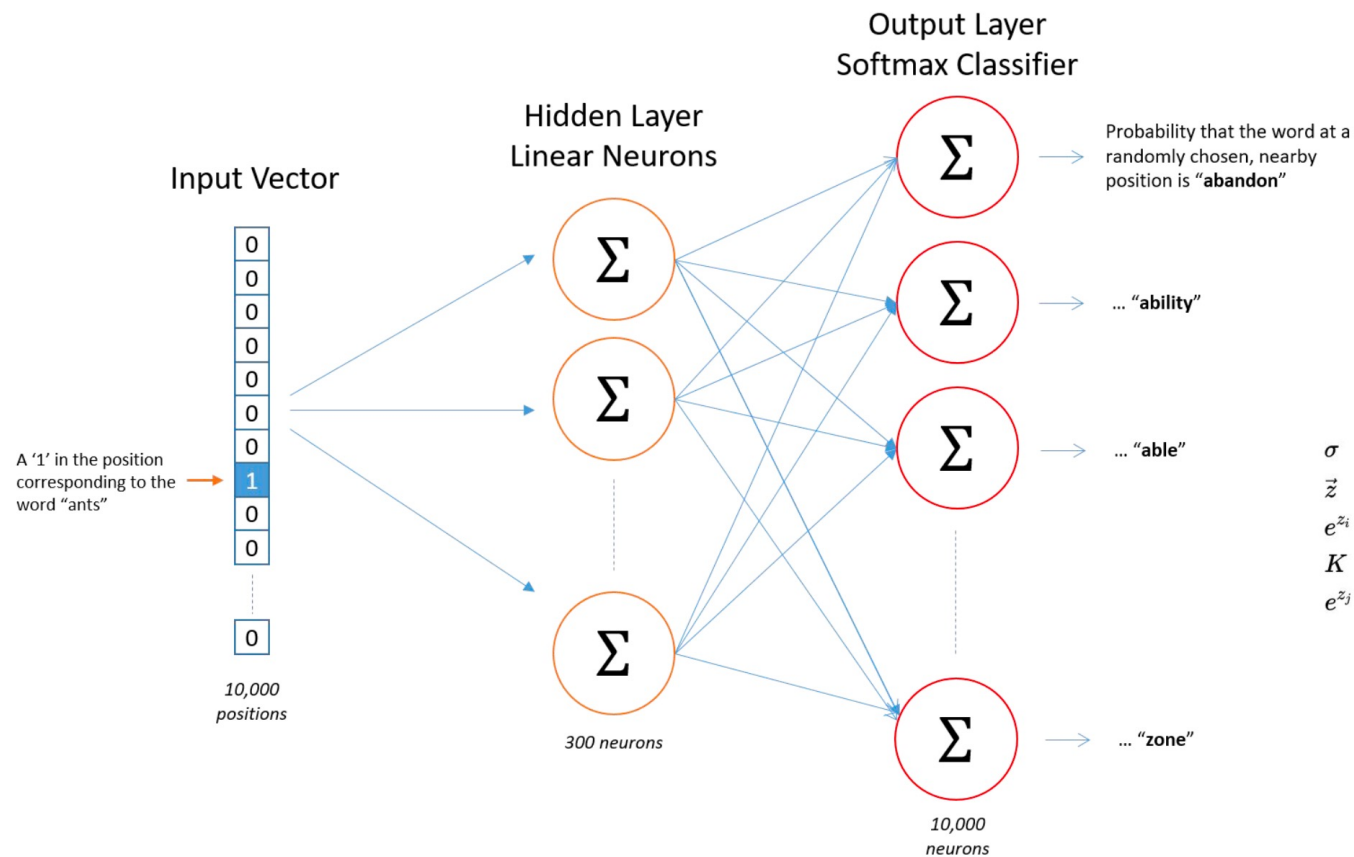
$$P("I", "like", "language", "processing" | "natural")$$

$$= P("I" | "natural") \times \\ P("like" | "natural") \times \\ P("language" | "natural") \times \\ P("processing" | "natural")$$

Architecture of the Skip-Gram Model



Architecture of the Skip-Gram Model (cont.)



$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = softmax

$\vec{z}$ = input vector

e^{z_i} = standard exponential function for input vector

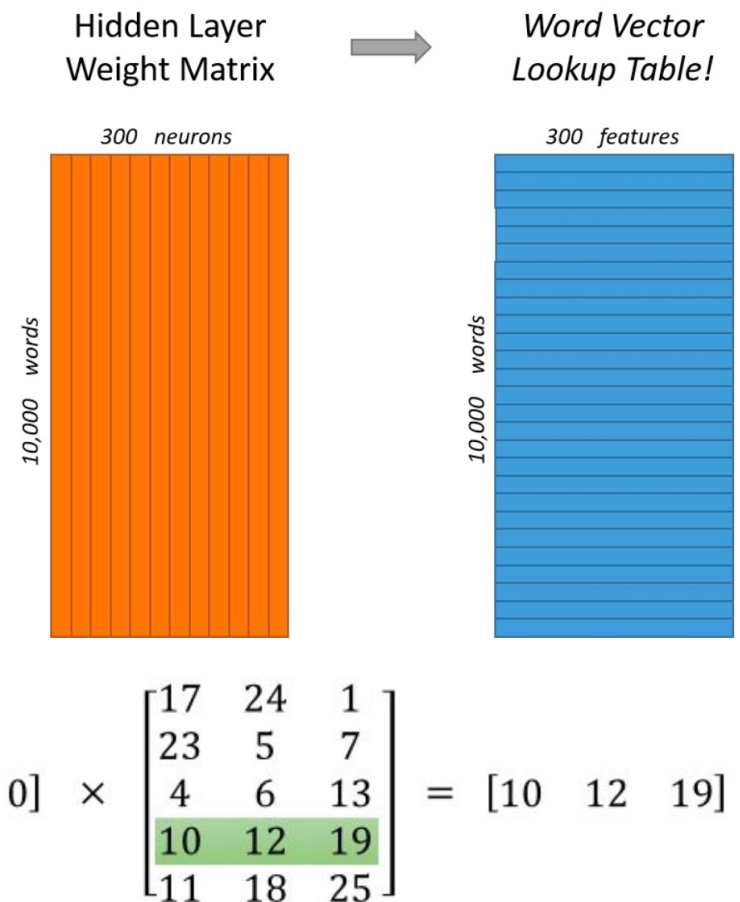
K = number of classes in the multi-class classifier

e^{z_j} = standard exponential function for output vector

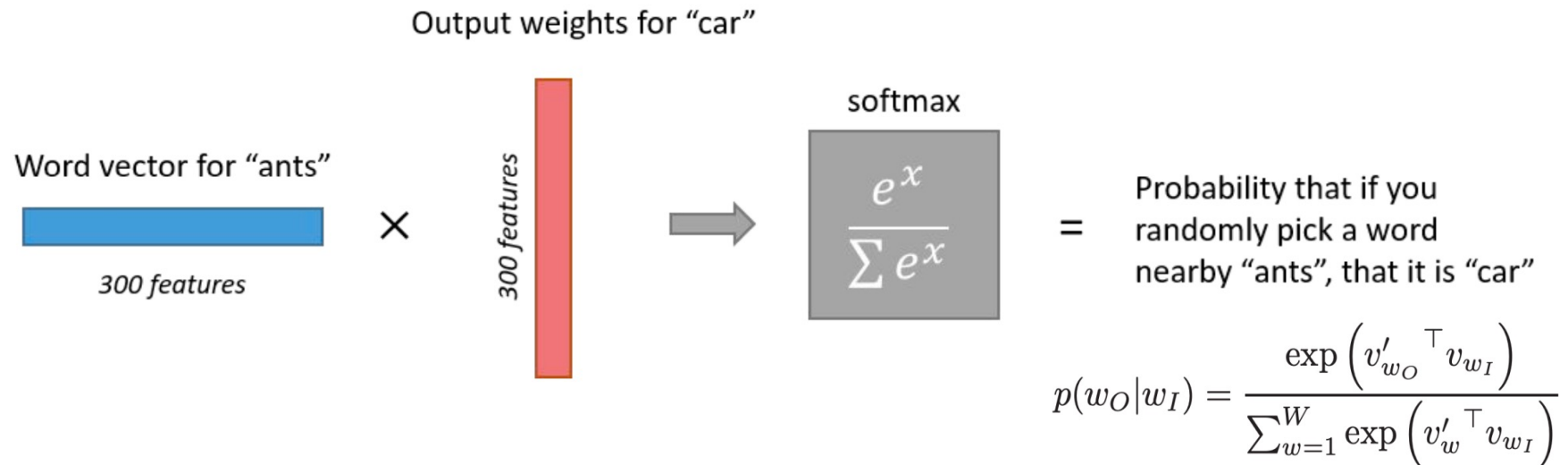


The hidden layer

- The hidden layer is going to be represented by a weight matrix
 - 10,000 rows (one for every word in our vocabulary)
 - 300 columns (one for every hidden neuron)[Google]
- The end goal of all of this is just to learn this hidden layer weight matrix
- The hidden layer of this model is just operating as a **lookup table**



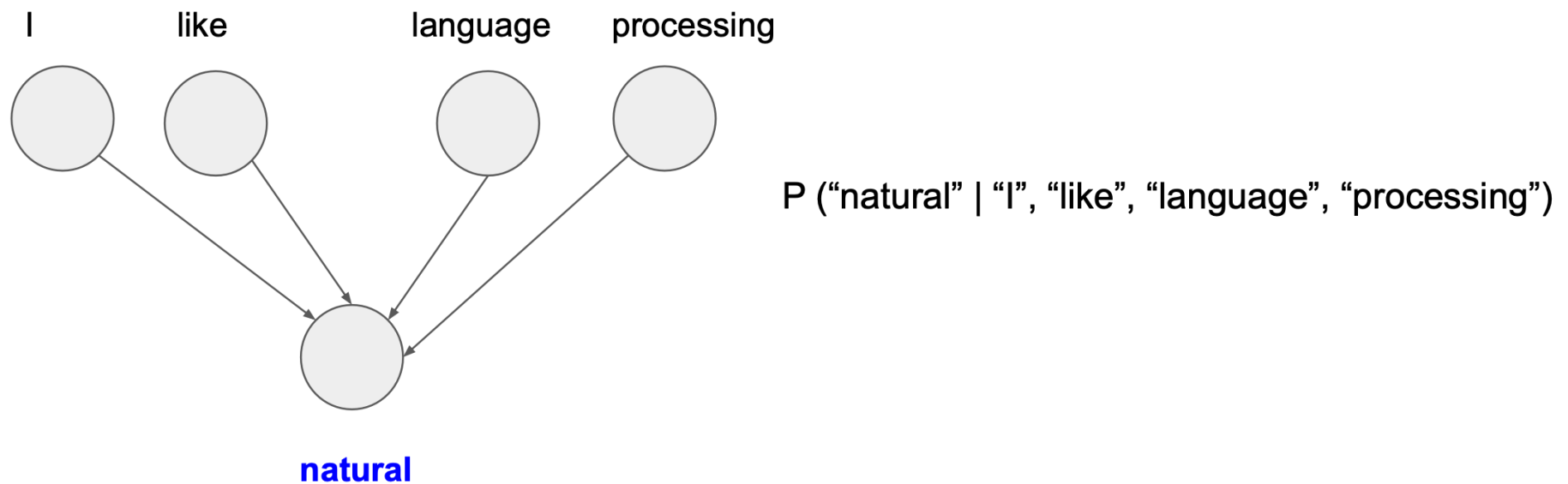
The output layer



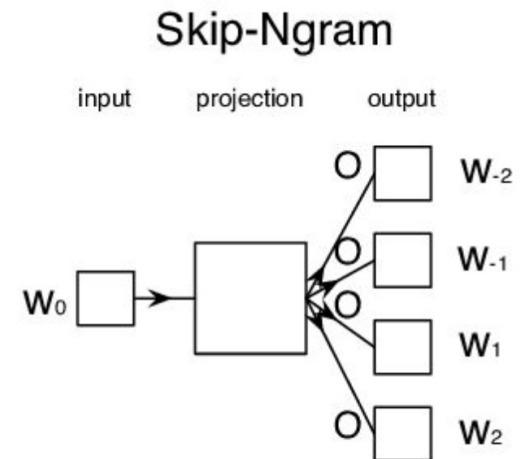
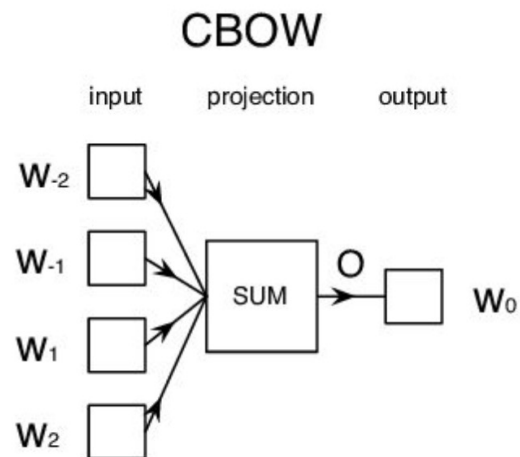
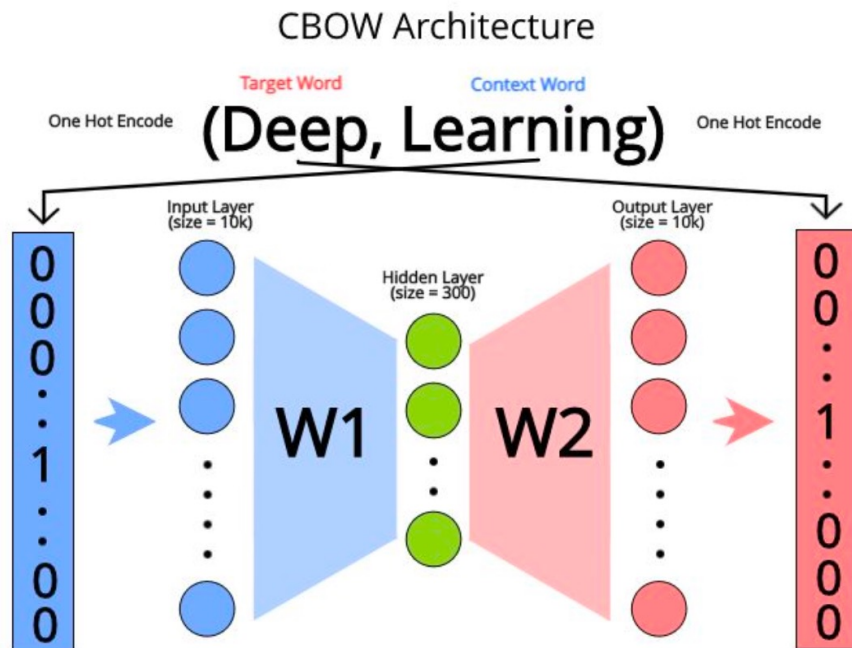
Softmax: each output neuron (one per word in our vocabulary) will produce an output between 0 and 1, and the sum of all these output values will add up to 1

Word2Vec (Continuous Bag of Words - CBOW model)

- The center word is generated based on the context words



Architecture of the CBOW model

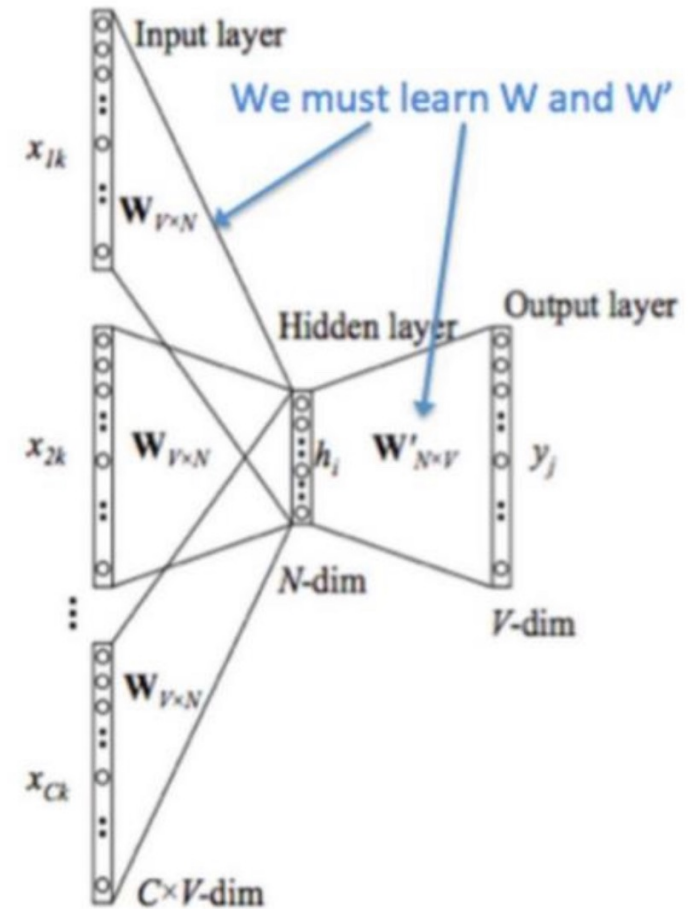


Architecture of the CBOW model

Input : x_0, x_1, x_3, x_4 output : x_2

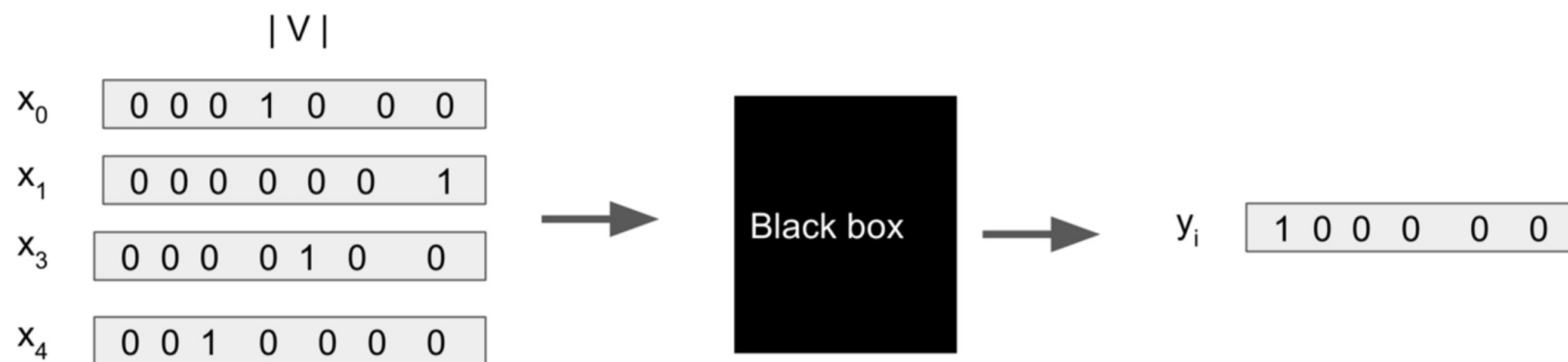
“The Cat Chills on a mat”

x_0 x_1 x_2 x_3 x_4 x_5



Input layer and Output layer

- $|V|$ is the size of the vocabulary corpus
- x_i represents the one-hot vector of the i^{th} word
- y_i represents the one-hot vector of the expected word



Output layer

$$y_i \quad \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$Z \quad \begin{bmatrix} 32 & 14 & 23 & 0.22 & 2 & 14 & 55 & 19 \end{bmatrix}$$

$$y^{\wedge} \quad \begin{bmatrix} 0.7 & 0.1 & 0.02 & 0.08 & 0 & 0 & 0.1 \end{bmatrix}$$

$$y^{\wedge} = \text{softmax} (Z) \quad \sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

$y_i \in \mathbb{R}^{|V| \times 1}$ là biểu diễn one-hot cho từ đúng, ở giữa (từ mong muốn)



Issues of this design

- For each training sample, only the **weights** corresponding to the **target word** might get a **significant update**
- While training a neural network model, in each back-propagation pass we try to **update all the weights** in the hidden layer
- The weight corresponding to non-target words would receive a marginal or **no change**

Solution: try to reduce the number of weights updated for each training sample



Subsampling of frequent words

- Frequent words can easily occur hundreds of millions of times (e.g., “in”, “the”, and “a”)
- **Observation**: the vector representations of frequent words do not change significantly after training on several million examples
- **Subsampling** is to sample the infrequent words more often than the frequent words to create new training data.
- The probability that we cut the word is related to the word's frequency

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

where t is a custom threshold (usually, a small value such as 0.0001)



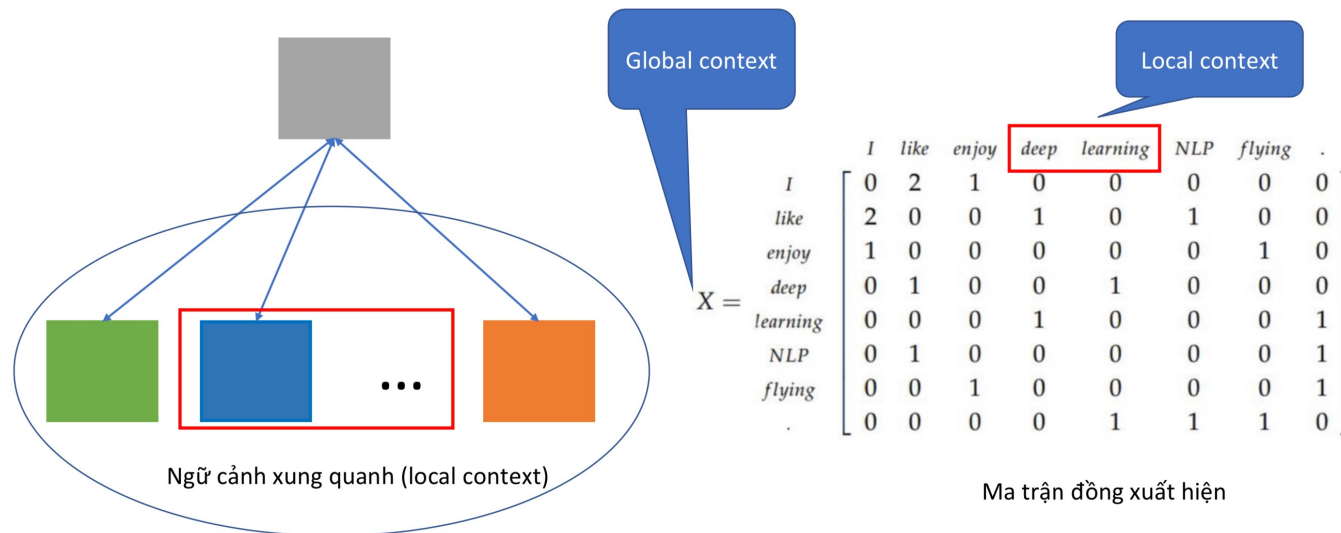
Negative sampling

- Instead of trying to predict the probability of being a nearby word for all the words in the vocabulary, we try to predict the probability that our training sample words are neighbors or not
- **Maximizing** the similarity of the words in the same context and **minimizing** it when they occur in different contexts
- Simplify the problem by randomly selecting a few “**negative**” words k to update the weights for

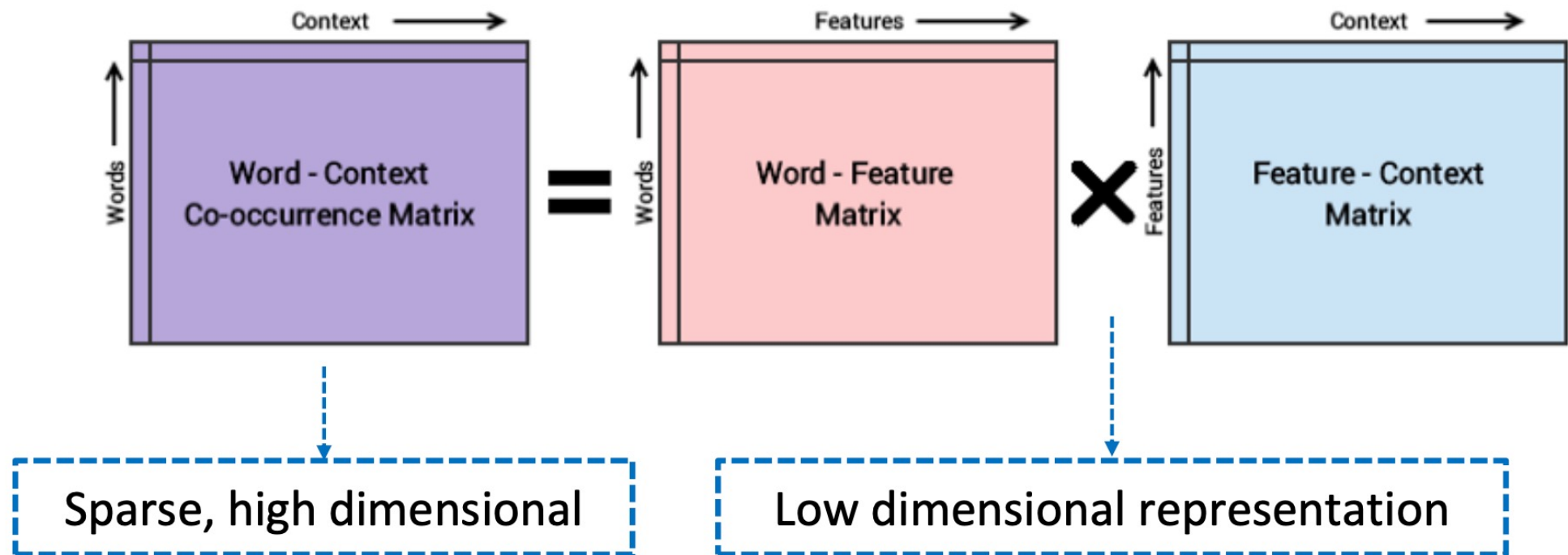


GLOVE – Global vector for word representation

- 2 assumptions:
 - Global context (global matrix factorization)
 - Co-occurrence of words through documents
 - Local context
 - A pre-fixed size slide window



GLOVE



GLOVE

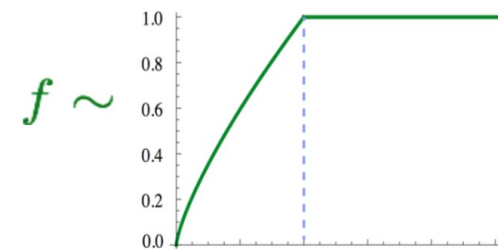
- Given a corpus having V words, the co-occurrence matrix X will be a $V \times V$ matrix, where the i th row and j th column of X , X_{ij} denotes how many times word i has co-occurred with word j

A: Log-bilinear model: $w_i \cdot w_j = \log P(i|j)$

with vector differences $w_x \cdot (w_a - w_b) = \log \frac{P(x|a)}{P(x|b)}$

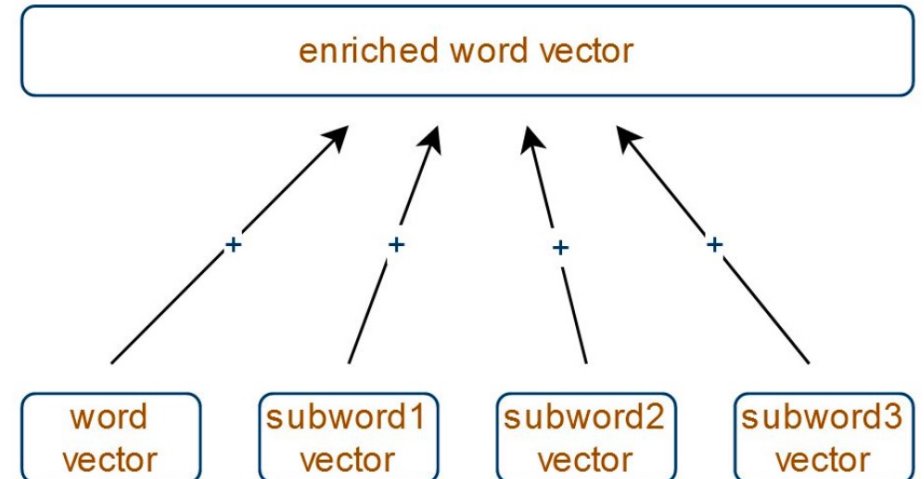
Loss:
$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

- Fast training
- Scalable to huge corpora



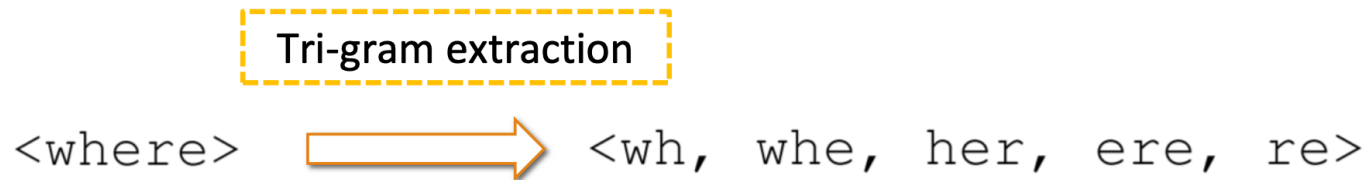
FastText

- Issues with GloVe and Word2Vec:
 - Out of Vocabulary words
 - An embedding is created for each word
 - Cannot handle any words it has not encountered during its training
- For words with same radicals such as “eat” and “eaten”, word2vec does not do any parameter sharing
- FastText n-gram embedding model:



Subword Generation

- For a word, we generate character n-grams of length 3 to 6 present in it



- Two-step vector representation updating:
 - First, the embedding for the **center word** is calculated by taking a **sum** of vectors for the **character n-grams** and the whole word itself
 - For the actual **context words**, we directly take their word vector from the embedding table **without** adding the **character n-grams**

FastText Training

FastText improves upon Word2Vec by incorporating subword information into word embedding

Word2Vec probability expression

$$p(w_O|w_I) = \frac{\exp\left(v'_{w_O}{}^\top v_{w_I}\right)}{\sum_{w=1}^W \exp\left(v'_w{}^\top v_{w_I}\right)}$$

Diagram illustrating the FastText probability expression and its components:

- The numerator term $v'_{w_O}{}^\top v_{w_I}$ is linked to the **N-gram embedding** box.
- The summation term $\sum_{g \in \mathcal{G}_w} \mathbf{z}_g{}^\top \mathbf{v}_c$ is linked to the **Represent a word by the sum of the vector representations of its n-grams** box.

Python Code Example – Load Word2Vec

```
from gensim.test.utils import common_texts
from gensim.models import Word2Vec

print("Sentences on Which We are gonna train our CBOW Word2Vec Model:\n")
print(common_texts)

Our_CBOV_Word2Vec_Model = Word2Vec(common_texts, vector_size = 10, window = 5, min_count
= 1, workers = 8, sg = 0)
Our_CBOV_Word2Vec_Model.save("Our_CBOV_Word2Vec_Model.w2v")
print("Model Saved")

Our_CBOV_Word2Vec_Model.wv.most_similar('human', topn = 5)
```



Python Code Example – Load FastText

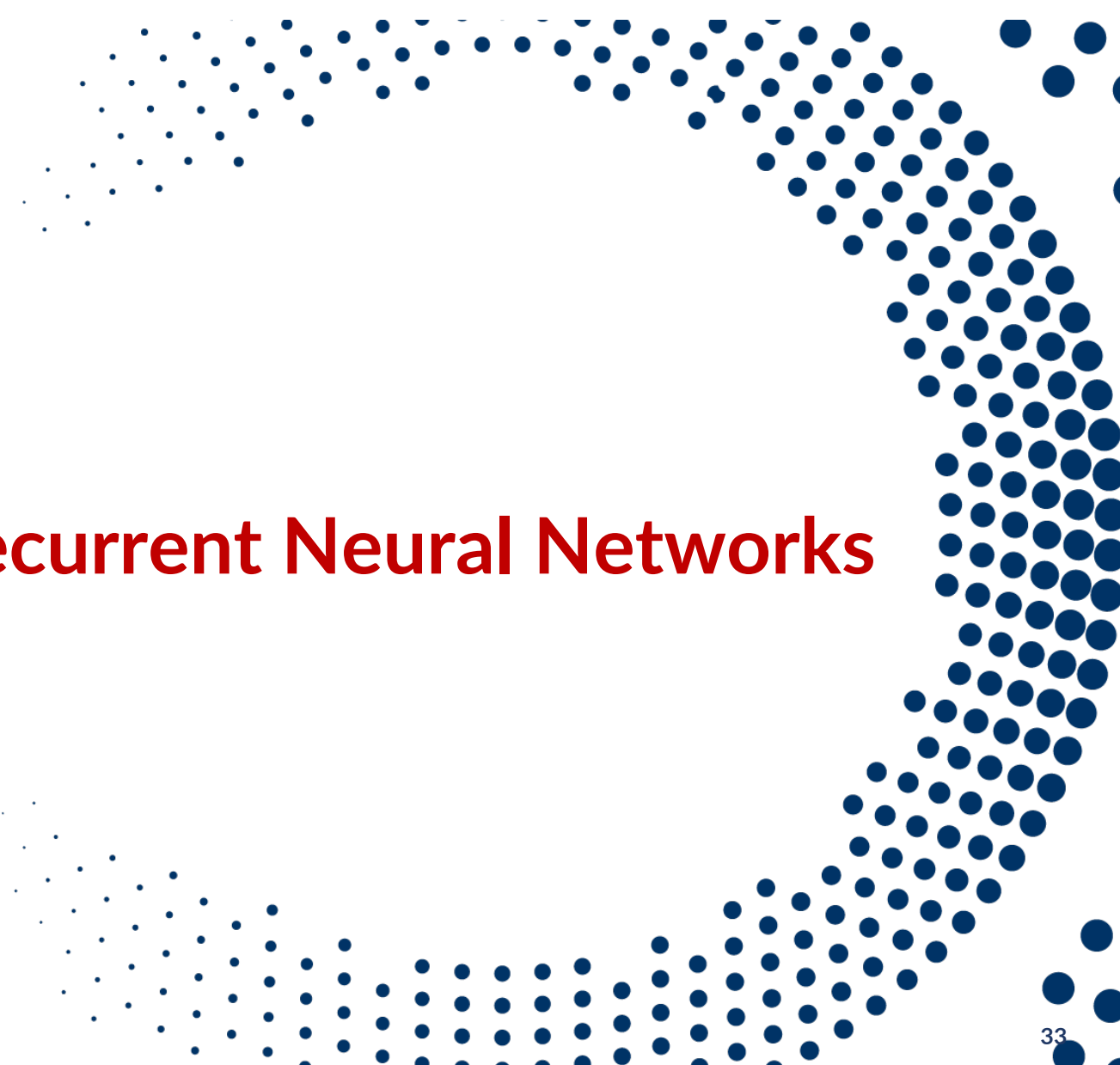
```
from gensim.models.fasttext import load_facebook_model
from gensim.models import FastText, KeyedVectors

fasttext_model = KeyedVectors.load_word2vec_format('../input/fasttext-wikinews/wiki-news-300d-1M.vec')

# fasttext_model = FastText.load_fasttext_format('../input/fasttext-crawl-300d-2m/crawl-300d-2M.vec')
print("Most similar words to word 'human' : ")
fasttext_model.most_similar('human')

print("Glove Word Embeddings of word 'human' ")
fasttext_model['human']
```

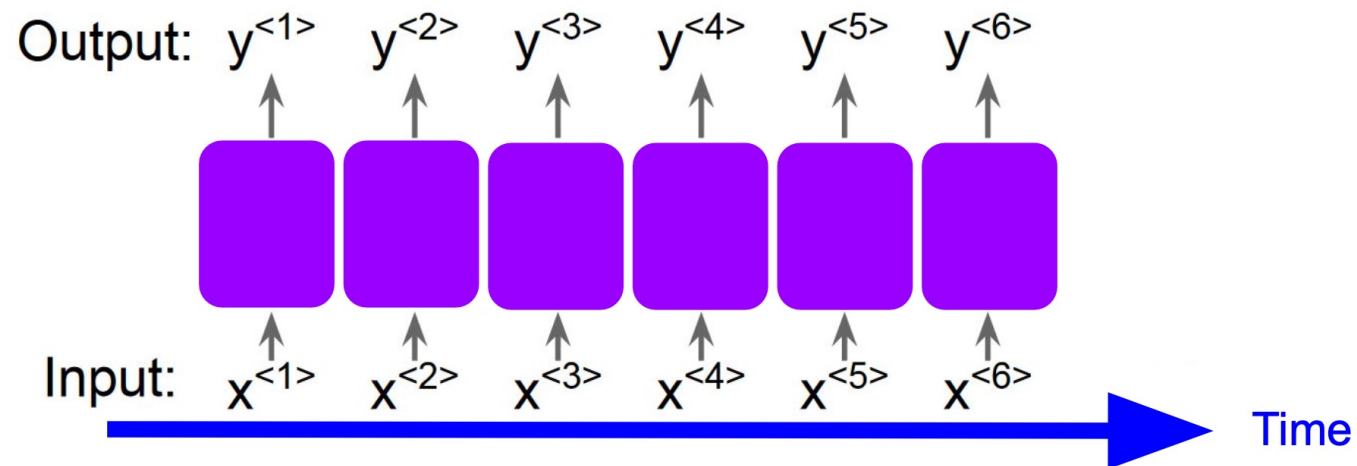




Section 2 – Recurrent Neural Networks

ONE LOVE. ONE FUTURE.

Sequential Data



- The movie he has **not** seen is good
- The movie he has seen is **not** good

rNNs

Feedforward nets can only handle inputs and outputs that have a fixed size

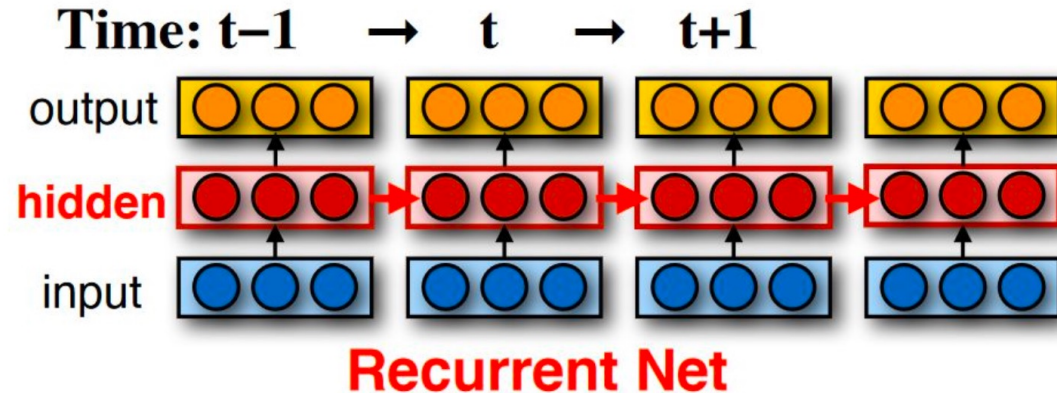
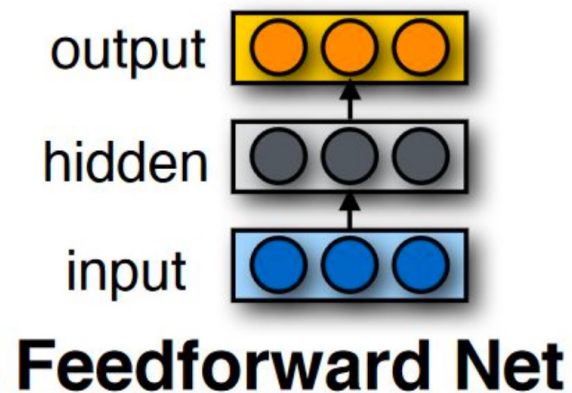
Recurrent Neural Nets (RNNs) handle variable length sequences (as input and as output)

There are 3 main variants of RNNs, which differ in their internal structure:

- Basic RNNs (Elman nets),
- Long Short-Term Memory cells (LSTMs)
- Gated Recurrent Units (GRUs)

Basic RNNs

- Generate a sequence of T outputs by running a variant of a feedforward net T times
- The hidden state computed at the previous step ($h^{(t-1)}$) is fed into the hidden state at the current step ($h^{(t)}$)



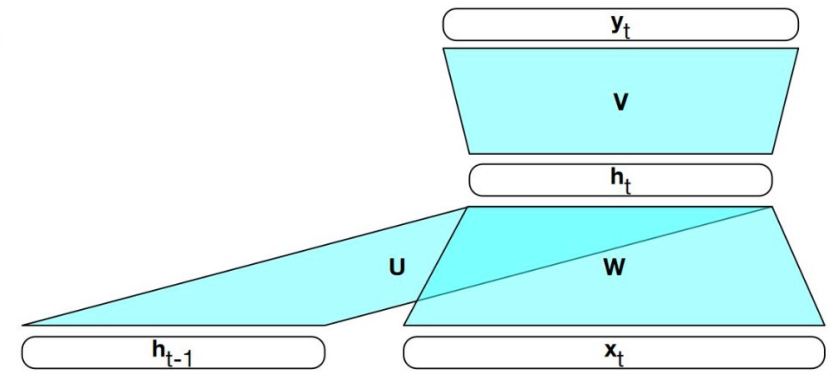
Structure of an RNN

To compute an output y_t for an input x_t :

- Multiply the input x_t with the weight matrix W , and the hidden layer from the previous time step h_{t-1} with the weight matrix U
- $W \in \mathbb{R}^{d_h \times d_{in}}$, $U \in \mathbb{R}^{d_h \times d_h}$, and $V \in \mathbb{R}^{d_{out} \times d_h}$

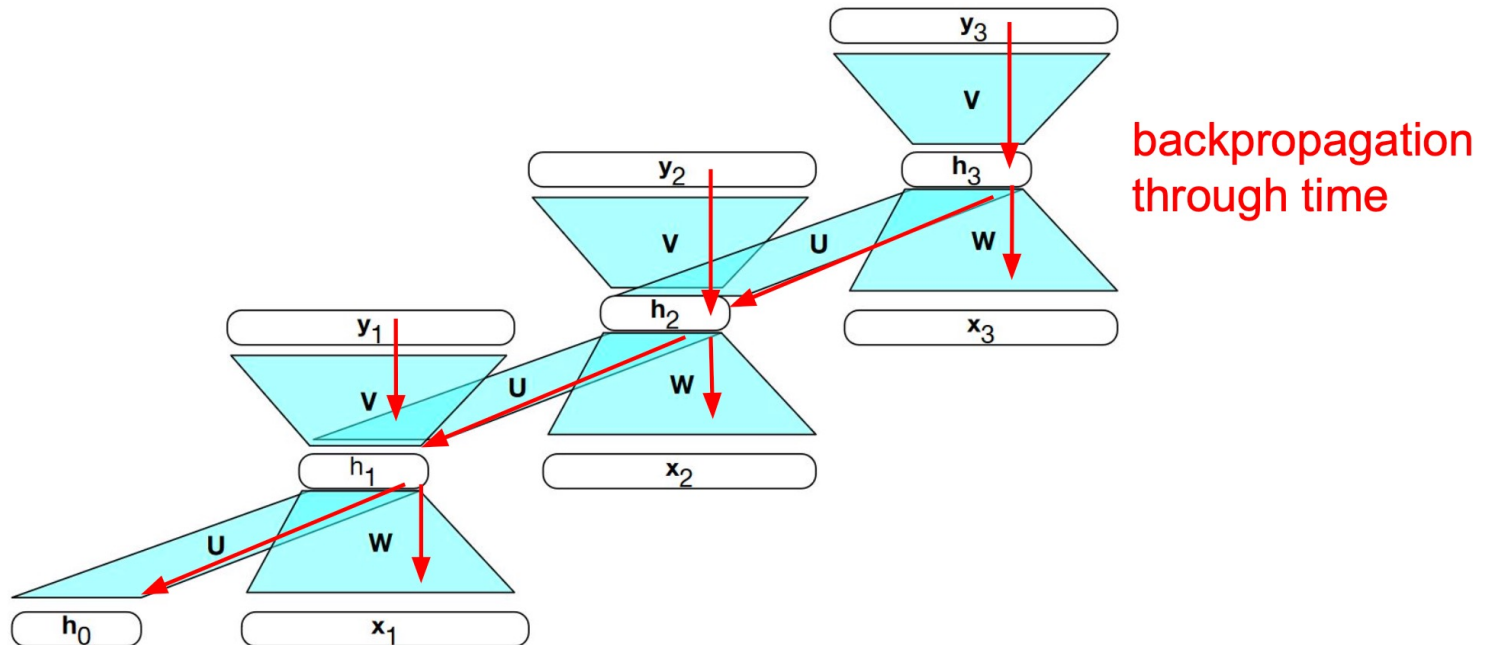
function FORWARDRNN($\mathbf{x}, network$) **returns** output sequence $\mathbf{y}$

```
 $\mathbf{h}_0 \leftarrow 0$   
for  $i \leftarrow 1$  to LENGTH( $\mathbf{x}$ ) do  
     $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$   
     $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$   
return  $\mathbf{y}$ 
```



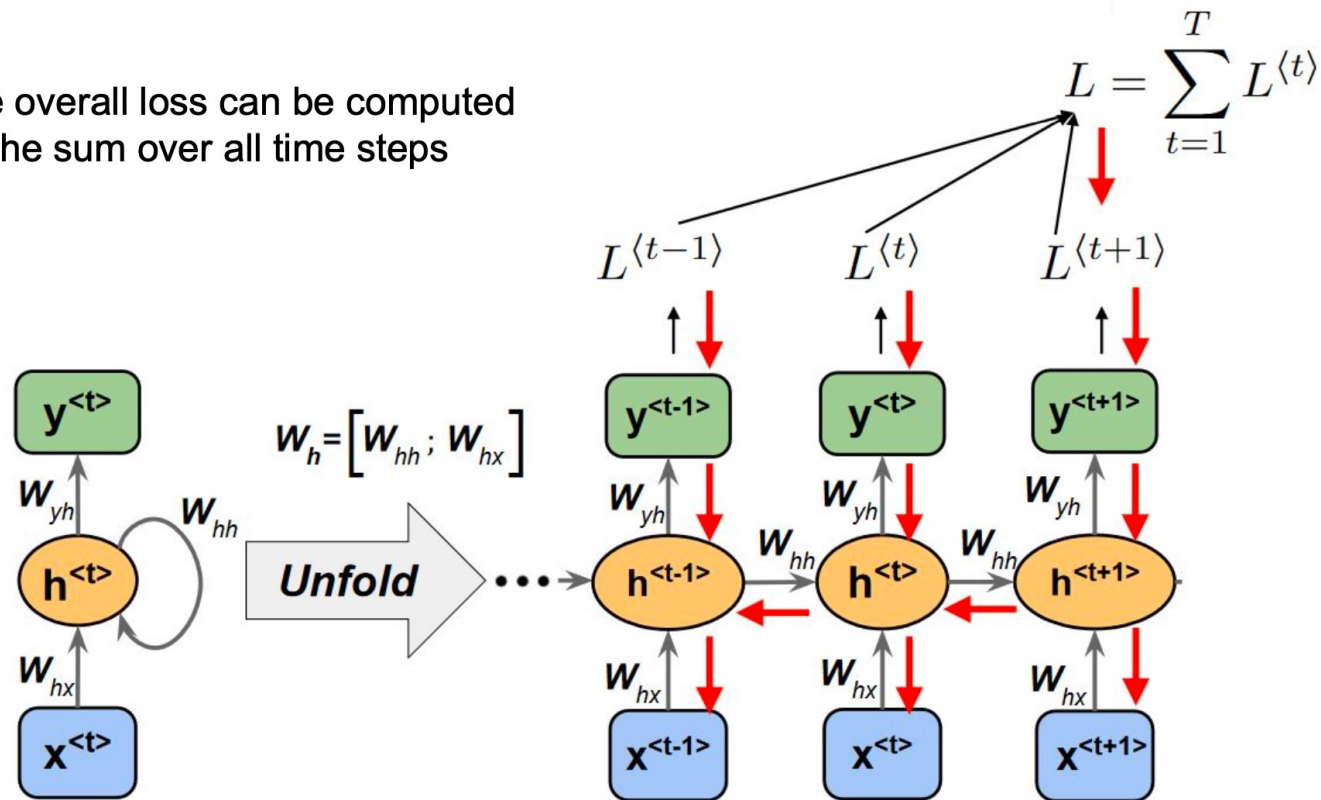
Training

- Training goal: estimate parameter values for U , V , W
- Use same loss as for feedforward language models
- Given unrolled network, run forward and backpropagation algorithms as usual



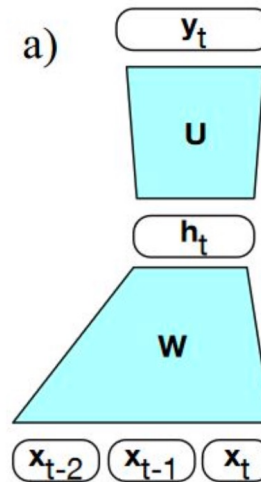
Backpropagation through time

The overall loss can be computed as the sum over all time steps

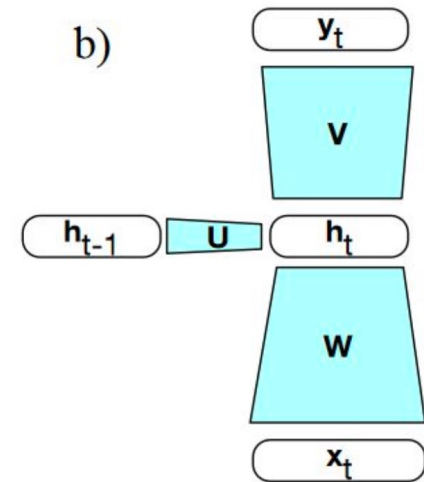


rNNs as Language Models

- Language Models predict the next word in a sequence given some preceding context
- RNN language models process the input sequence one word at a time, attempting to **predict the next word** from the **current word** and the **previous hidden state**



a) Feedforward neural



(b) RNN language models

Forward Inference in an rNN Language Model

The input sequence $X = [x_1; \dots; x_t; \dots; x_N]$ consists of a series of word embeddings each represented as a one-hot vector of size $|V| \times 1$, and the output prediction y_t , is a vector representing a probability distribution over the vocabulary

At time t :

$$\begin{aligned} \mathbf{e}_t &= \mathbf{E}\mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \\ \mathbf{y}_t &= \text{softmax}(\mathbf{V}\mathbf{h}_t) \end{aligned}$$

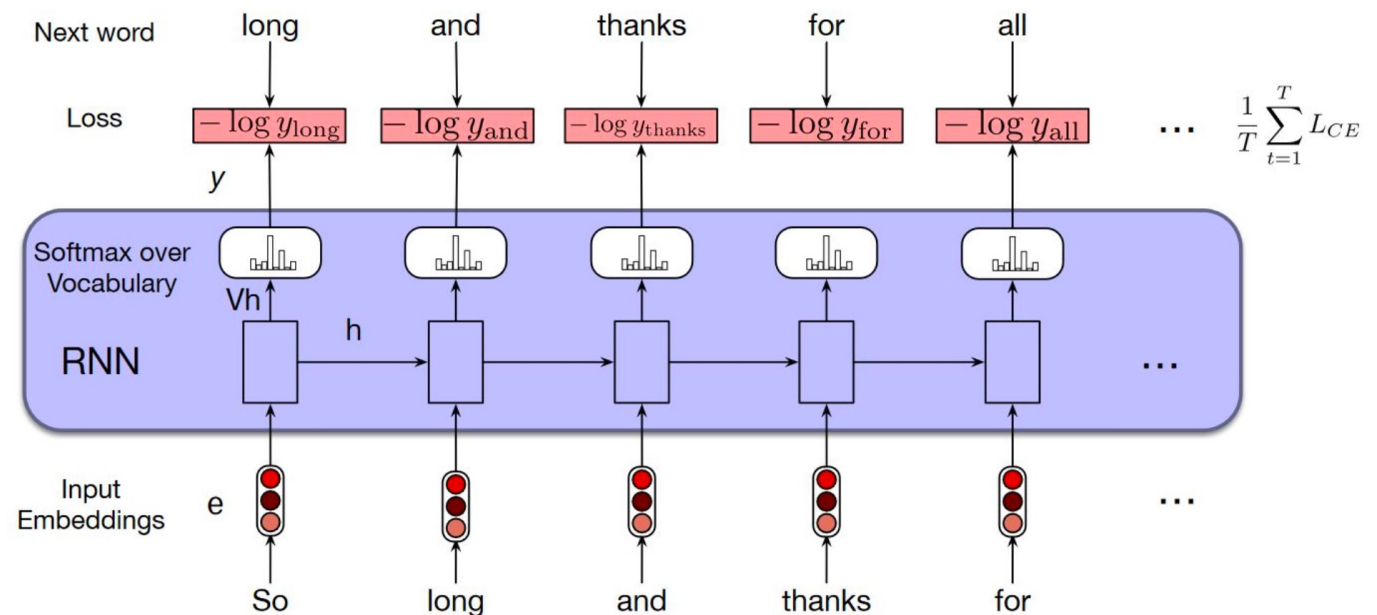
The vector resulting from $\mathbf{V}\mathbf{h}$ can be thought of as a set of scores over the vocabulary given the evidence provided in $\mathbf{h}$

The probability that a particular word i in the vocabulary is the next word is represented by $y_t[i]$, the i th component of $\mathbf{y}_t$:

$$P(w_{t+1} = i | w_1, \dots, w_t) = \mathbf{y}_t[i]$$

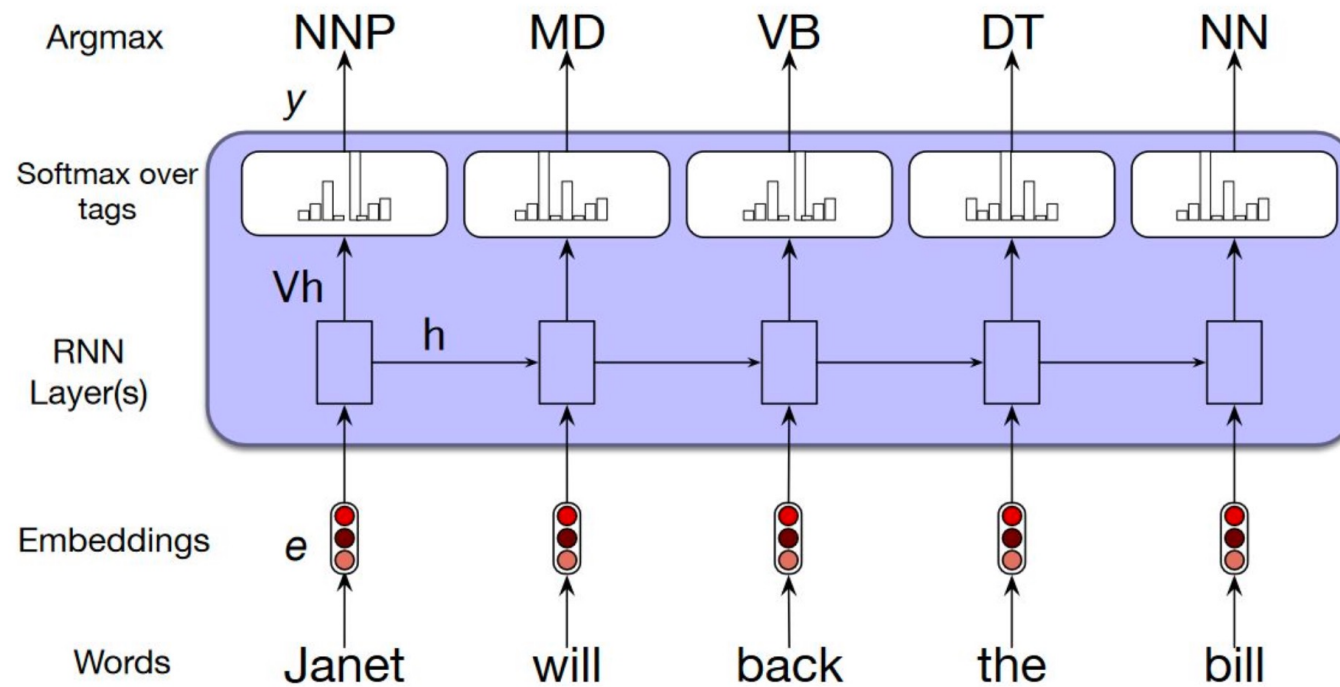
Training an rNN Language Model

- Self-supervised training (no labelled data)
- At a time step t , we take the model to predict the next word with a corpus of text as training data
- The objective of training is to **minimize** the **error** in predicting the true next word in the training sequence



Sequence Labeling

Forward inference over the input sequence; select the most likely tag from the softmax



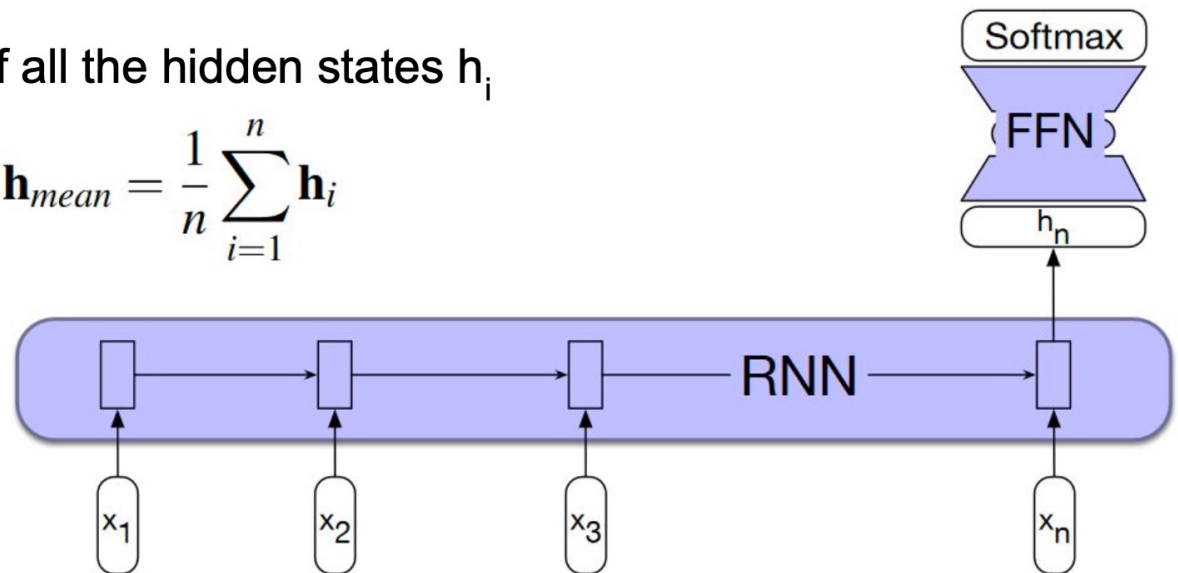
Sequence Classification

To apply RNNs in this setting, we pass the text to be classified through the RNN a word at a time, generating a new hidden layer at each time step

1. We can pass this representation h_n to a feedforward network that chooses a class via a softmax over the possible classes
2. Use some sort of pooling function of all the hidden states h_i

$$\mathbf{h}_{mean} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i$$

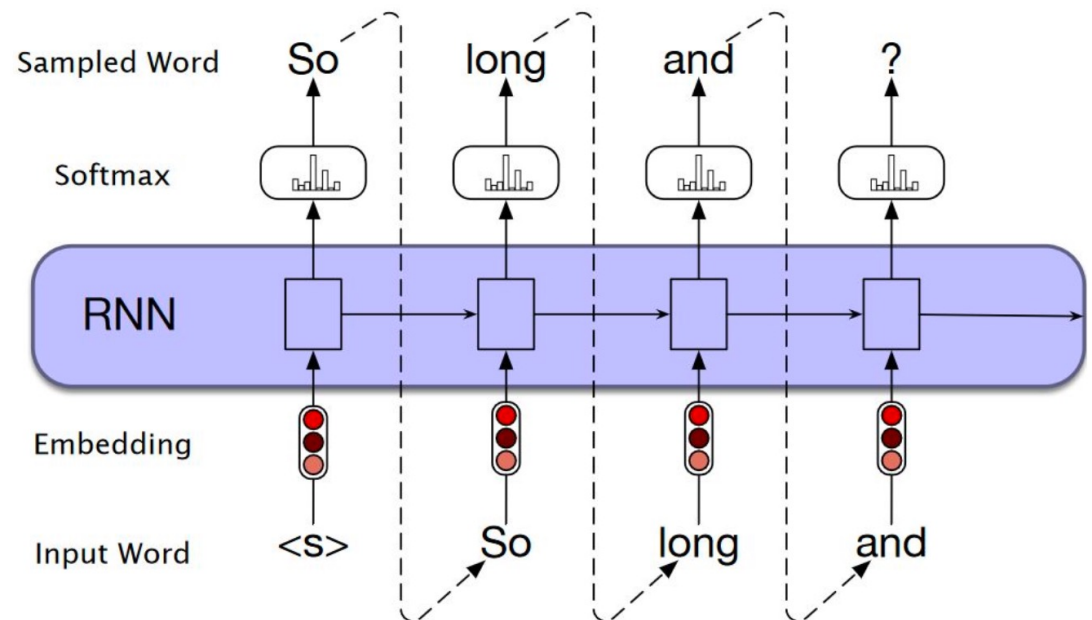
3. Take the element-wise max



Language Generation

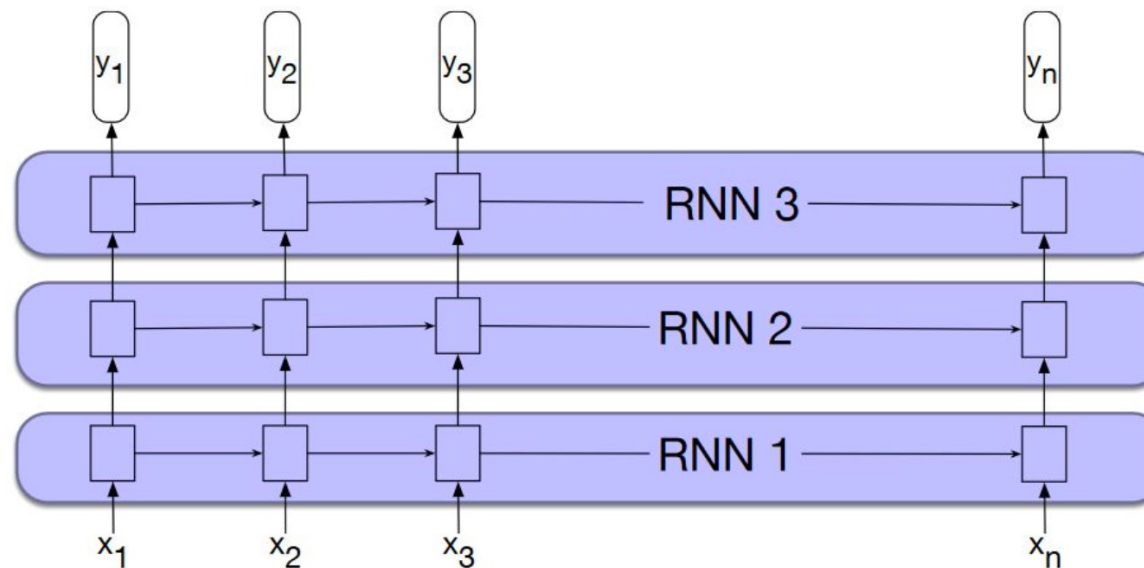
Autoregressive generation: Using a language model to incrementally generate words by repeatedly sampling the next word conditioned on our previous choices

1. Sample a word in the output from the softmax distribution that results from using the beginning of sentence marker, `<s>`, as the first input
2. Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion
3. Continue generating until the end of sentence marker, `</s>`, is sampled or a fixed length limit is reached



Stacked RNNs

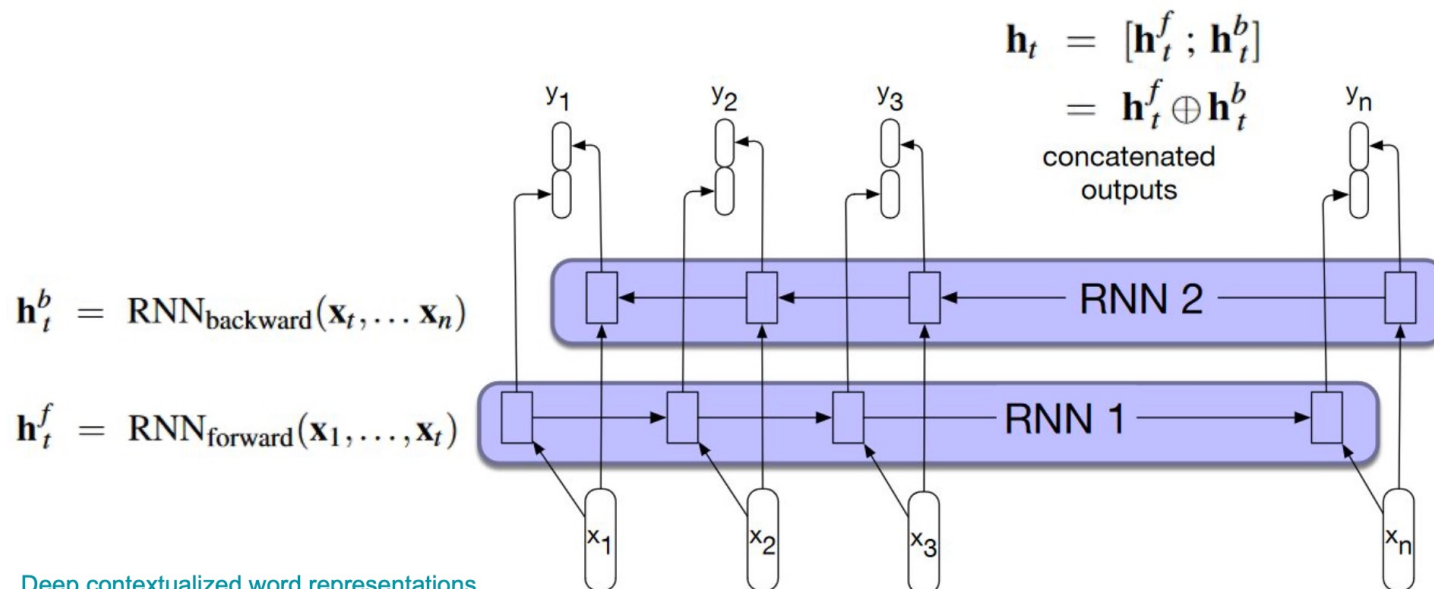
Stacked RNNs consist of multiple networks where the output of one layer serves as the input to a subsequent layer



Bidirectional RNNs

Combines two independent RNNs, one where the input is processed from the start to the end, and the other from the end to the start

Concatenate the two representations computed by the networks into a single vector that captures both the left and right contexts of an input at each point in time

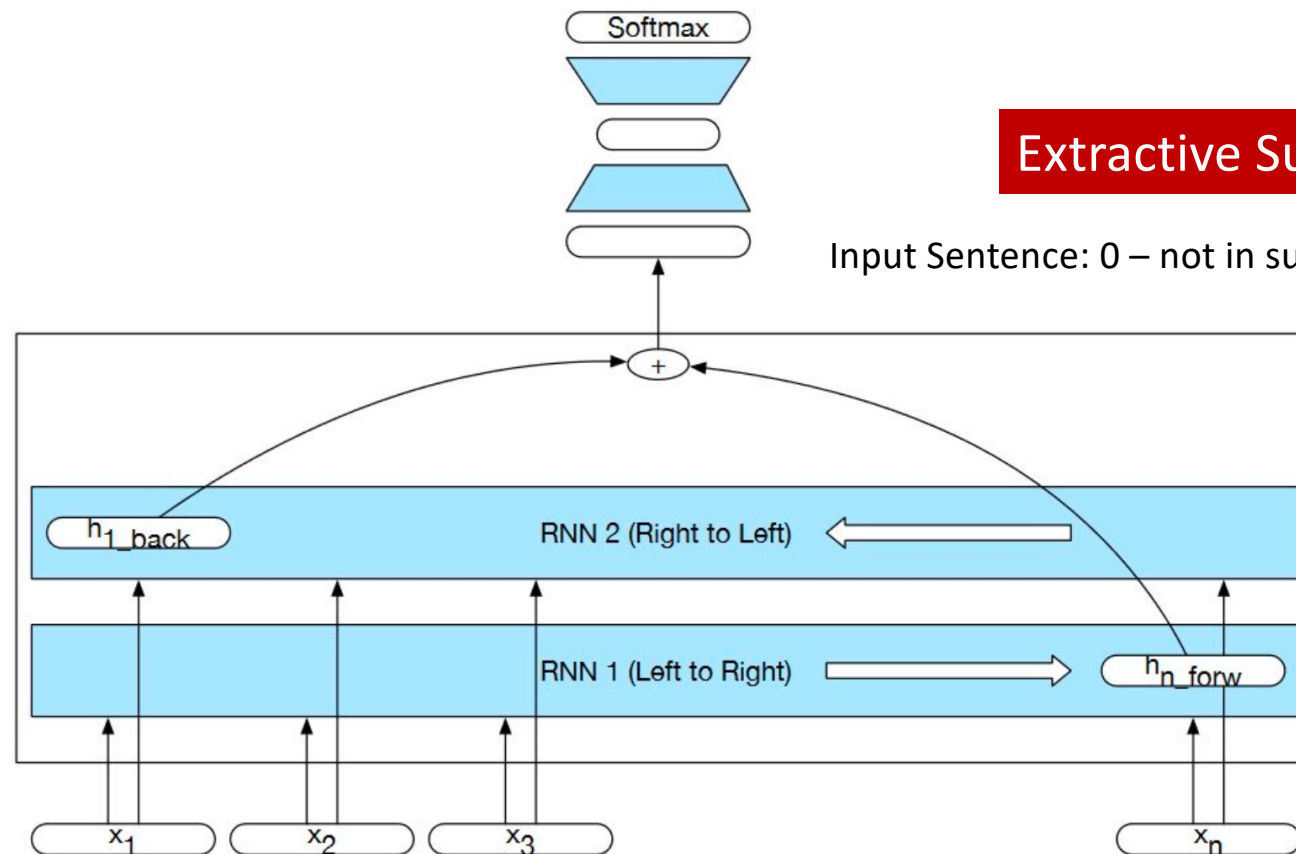


[Deep contextualized word representations](#)



TRƯỜNG ĐẠI HỌC BACH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Bidirectional RNNs for Sequence Classification



Vanishing gradient

In practice, it is quite difficult to train RNNs for tasks that require a network to make use of information distant from the current point of processing

- The information encoded in hidden states tends to be fairly local, more relevant to the most recent parts of the input sequence and recent decisions

The inability of RNNs to carry forward critical information

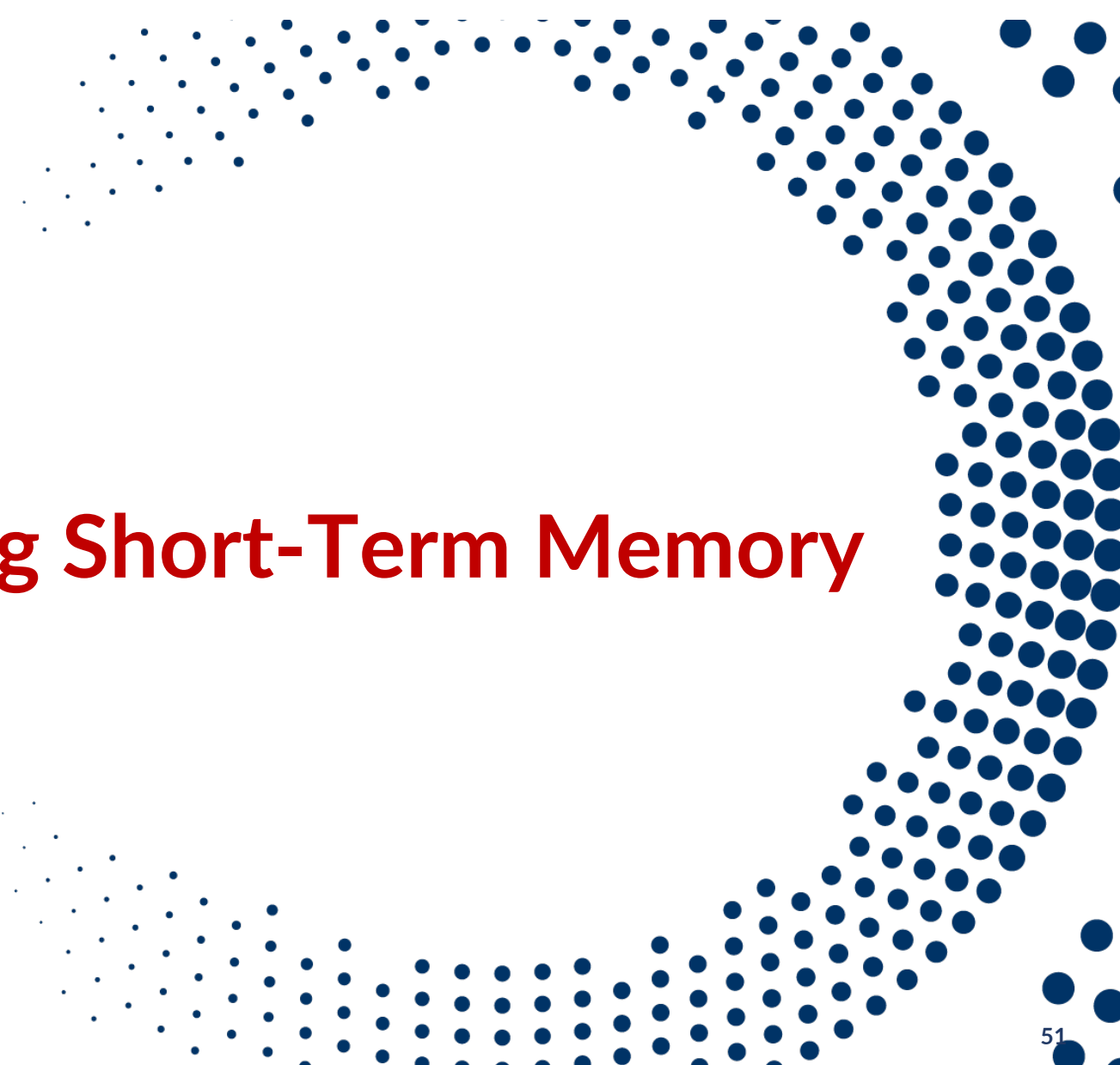
- With Tanh: Models suffer from the **vanishing gradient** problem
 - Gradients get smaller and approach zero, which eventually leaves the weights of the initial or lower layers nearly unchanged
- With ReLU: Models suffer from the **exploding gradient** problem
 - Gradients keep on getting larger as the backpropagation progresses



Solution

- Explicitly manage the task of maintaining relevant context over time
- Enable the network to learn
 - To forget information that is no longer needed
 - To remember information required for decisions still to come
- LSTM
- GRU





Section 3 - Long Short-Term Memory

ONE LOVE. ONE FUTURE.

LSTM – Long Shortterm Memory Network

- Solving the issue by:
 - Removing information no longer needed from the context
 - Adding information likely to be needed for later decision-making
- Make use of **gates** to control the flow of information into and out of the units that comprise the network layers
- Three different gates, input **add**, **forget** and, **output** that read in the previous hidden state and current input to decide **how much of the past** hidden and cell states to **keep**



Gates

The gates in an LSTM:

- each consists of a feedforward layer
- followed by a sigmoid activation function
- followed by a pointwise multiplication with the layer being gated

Forget gate: delete information from the context that is no longer needed

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t) \quad \mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$$

what information
needs to be
preserved for future
decisions

Input (add) gate: select the information to add to the current context

$$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t) \quad \mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t) \quad \mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t \quad \mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$$

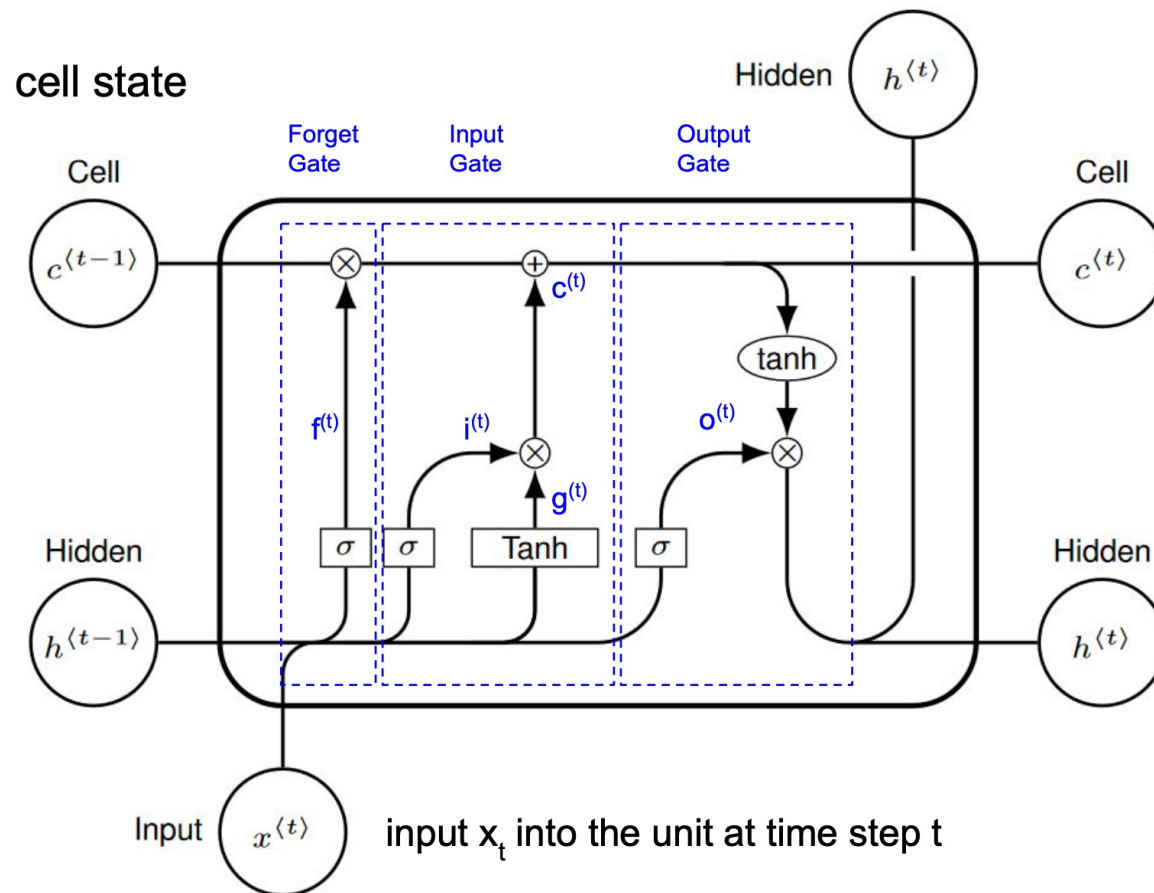
Output gate: decide what information is required for the current hidden state

$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t) \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

What information is
required for the
current hidden state



LSTM Unit



The gates in an LSTM:

- each consists of a feedforward layer
- followed by a sigmoid activation function
- followed by a pointwise multiplication with the layer being gated

$$i^{(t)} = \sigma(W_i x^{(t)} + U_i h^{(t-1)} + b_i)$$

$$f^{(t)} = \sigma(W_f x^{(t)} + U_f h^{(t-1)} + b_f)$$

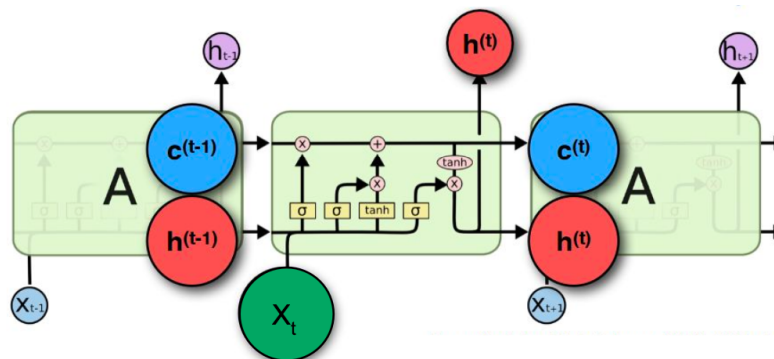
$$o^{(t)} = \sigma(W_o x^{(t)} + U_o h^{(t-1)} + b_o)$$

$$g^{(t)} = \tanh(W_g x^{(t)} + U_g h^{(t-1)} + b_g)$$

$$c^{(t)} = i^{(t)} \otimes g^{(t)} + c^{(t-1)} \otimes f^{(t)}$$

$$h^{(t)} = o^{(t)} \otimes \tanh(c^{(t)})$$

At time t in LSTM

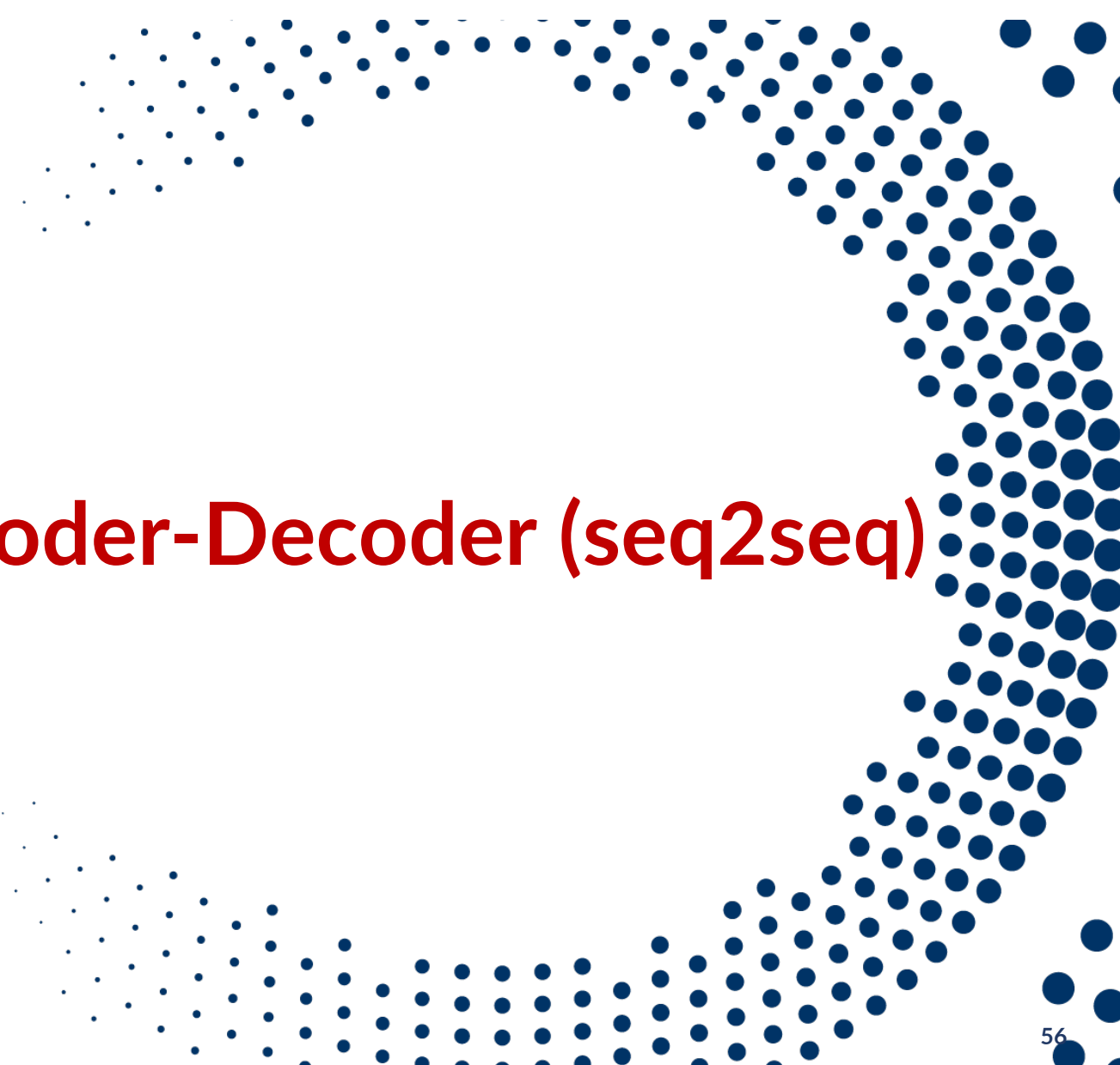


At time t , the LSTM cell reads in

- a c -dimensional **previous cell state vector** $c^{(t-1)}$
- a h -dimensional **previous hidden state vector** $h^{(t-1)}$
- a d -dimensional **current input vector** $x^{(t)}$

At time t , the LSTM cell returns

- a c -dimensional **new cell state vector** $c^{(t)}$
- a h -dimensional **new hidden state vector** $h^{(t)}$ (which may also be passed to an output layer)



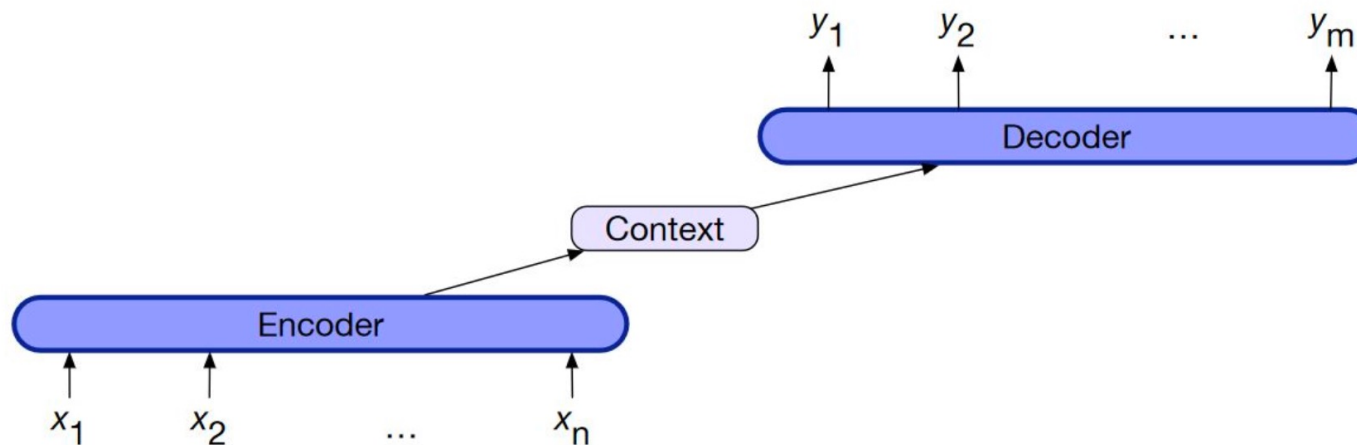
Section 4 - Encoder-Decoder (seq2seq)

ONE LOVE. ONE FUTURE.

Encoder-Decoder

The key idea underlying these networks is the use of an **encoder** network that takes an input sequence and creates a contextualized representation of it, often called the **context**

This representation is then passed to a **decoder** which generates a task-specific output sequence

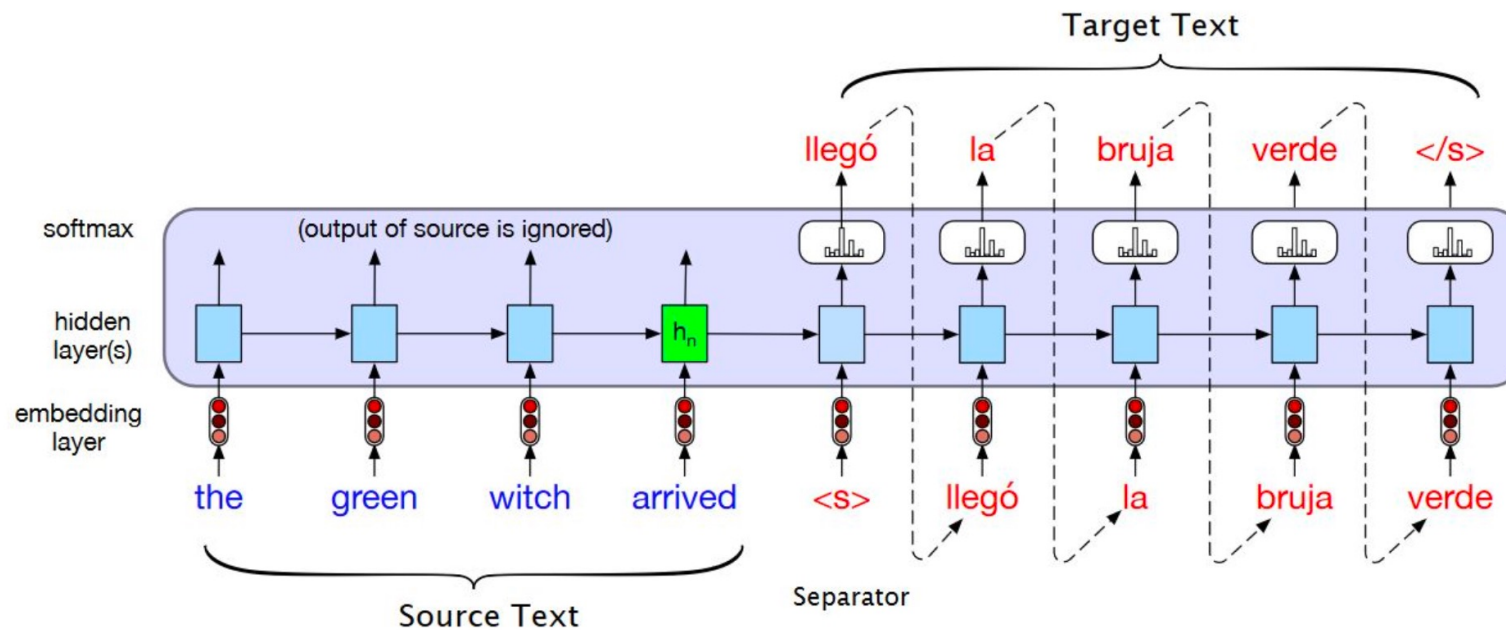


Three components of a seq2seq

1. An **encoder** that accepts an input sequence, x^n_1 , and generates a corresponding sequence of contextualized representations, h^n_1
 - LSTMs, convolutional networks, and Transformers can all be employed as encoders
2. A **context** vector, c , which is a function of h^n_1 , and conveys the essence of the input to the decoder
3. A **decoder**, which accepts c as input and generates an arbitrary length sequence of hidden states h^m_1 , from which a corresponding sequence of output states y^m_1 , can be obtained
 - Just as with encoders, decoders can be realized by any kind of sequence architecture

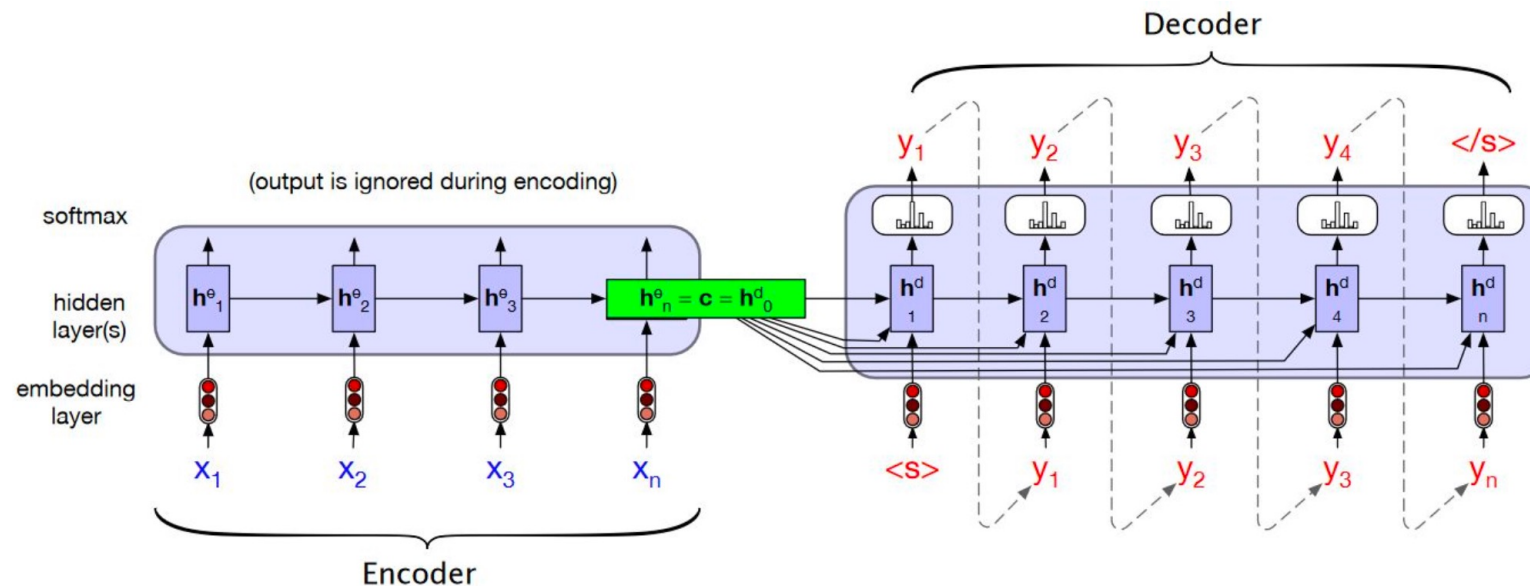


Seq2Seq for Translation



Similar to language modeling: add a sentence separation marker at the end of the source text concatenate the target text

Seq2Seq for Translation



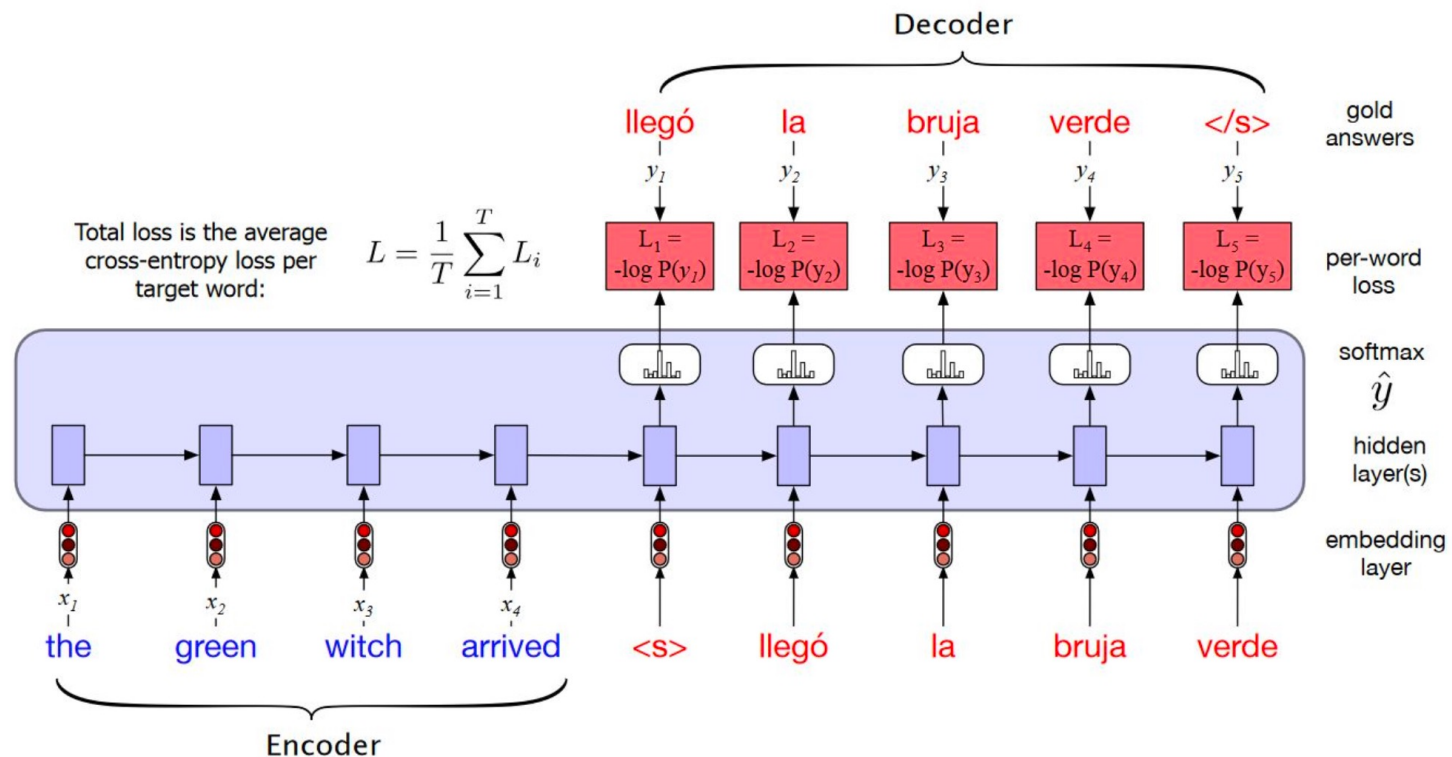
Encoder as stacked biLSTMs: the hidden states from top layers from the forward and backward passes are concatenated to provide the contextualized representations for each time step

Also make the context vector c available at each step in the decoding process by adding it as a parameter to the computation of the current hidden state

$$h_t^d = g(\hat{y}_{t-1}, h_{t-1}^d, c)$$

Training

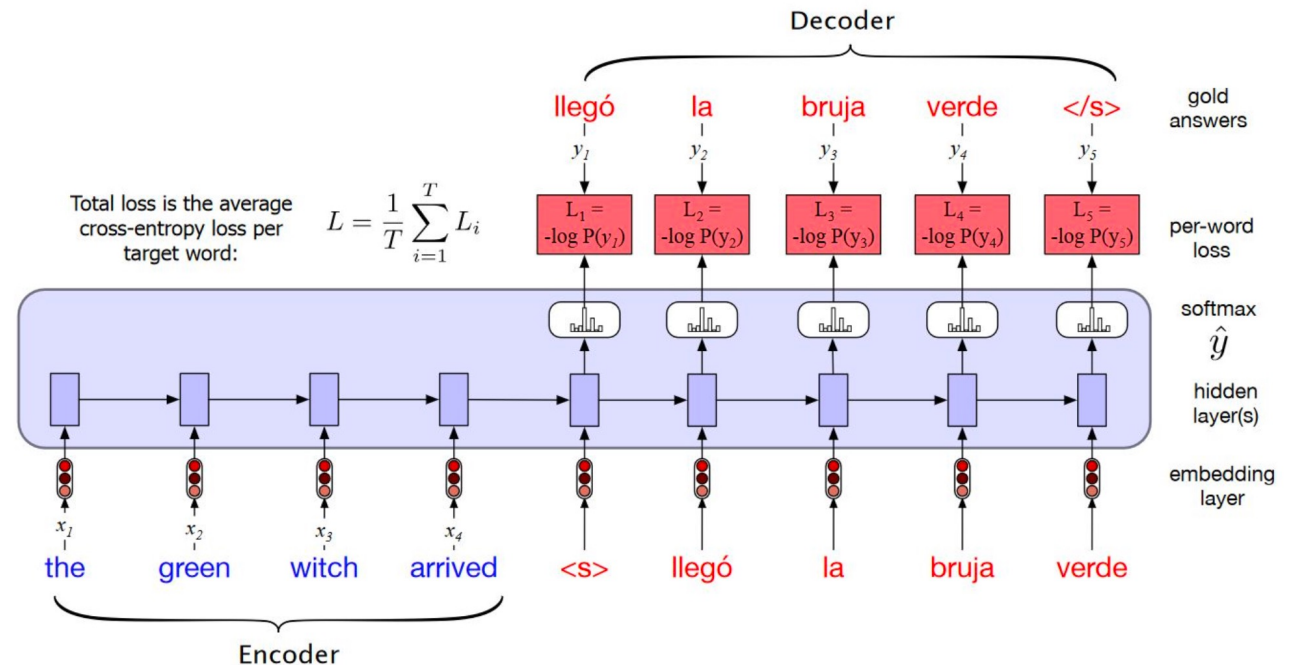
For Machine Translation, the training data typically consists of sets of sentences and their translations
Once we have a training set, the training itself proceeds as with any RNN-based language model



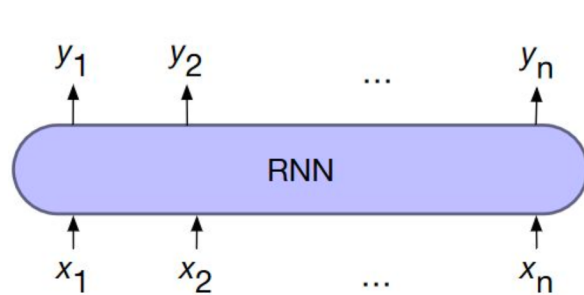
Inference

The decoder during inference uses its own estimated output as the input for the next time step x_{t+1}

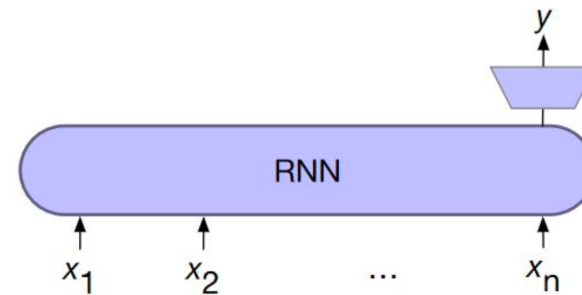
Teacher forcing: force the system to use the gold target token from training as the next input x_{t+1}



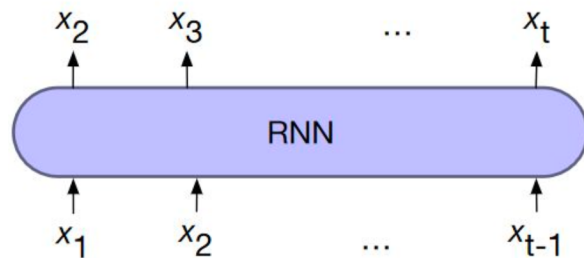
Summary 4 common architectures with rNNs



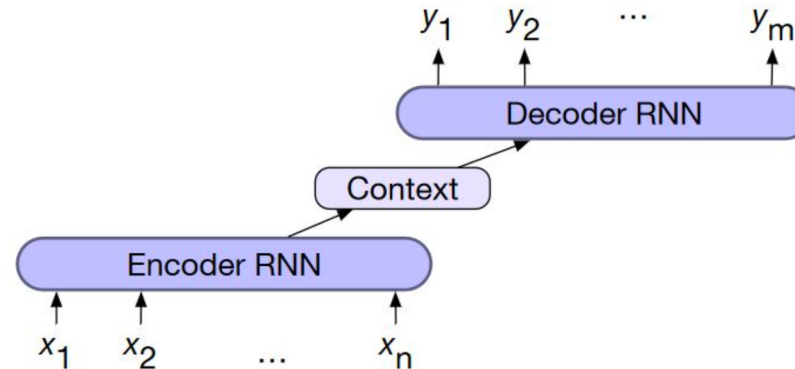
a) sequence labeling



b) sequence classification



c) language modeling



d) encoder-decoder



Section 5 - Attention

ONE LOVE. ONE FUTURE.

Back to Encoder-Decoder Model

Context vector is $\mathbf{h}_n$, the hidden state of the last (nth) time step of the source text

The only thing the decoder knows about the source text is the context vector

- Information at the beginning of the sentence, especially for long sentences, may not be equally well represented in the context vector

Solution: **Attention**

Allowing the decoder to get information from all the hidden states of the encoder, not just the last hidden state!

Context vector $\mathbf{c}$ is a single vector that is a function of the hidden states of the encoder

$$\mathbf{c} = f(\mathbf{h}_1^e \dots \mathbf{h}_n^e)$$



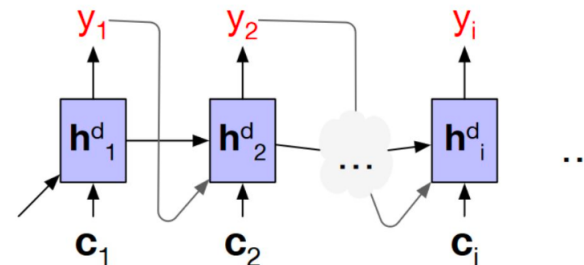
New context vector

Create the single fixed-length vector c by taking a weighted sum of all the encoder hidden states

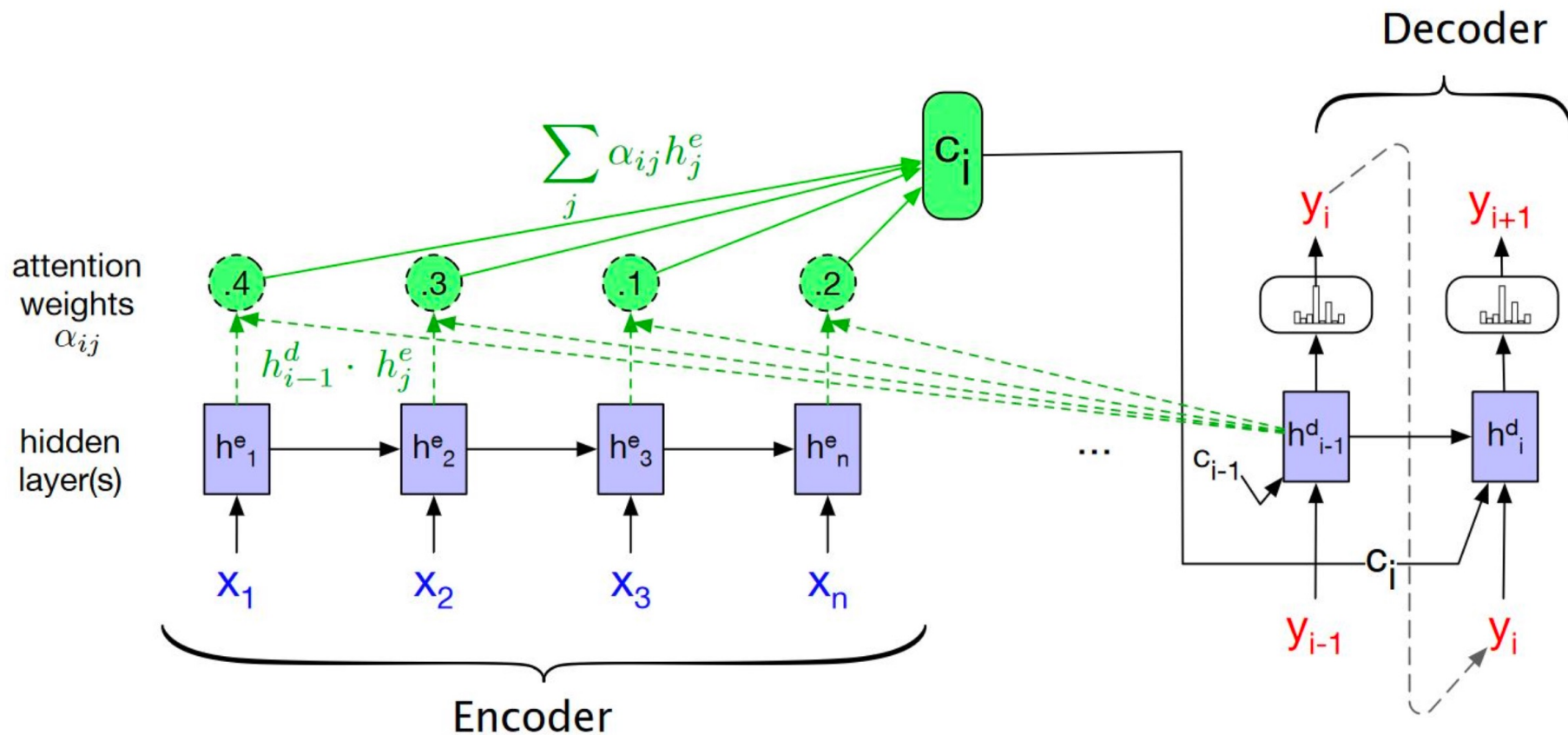
The weights focus on ('attend to') a particular part of the source text that is relevant for the token the decoder is currently producing

For each decoding step i , a new context vector c_i is created and the hidden state is calculated as:

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



Encoder-Decoder with Attention



Calculate c_i

How relevant each encoder state is to the decoder state captured in h_{i-1}^d

- At each state i during decoding, calculate score (h_{i-1}^d, h_j^e) for each state j in encoder
- Simplest score: dot-product attention

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$

The vector of these scores across all the encoder hidden states gives us the relevance of each encoder state to the current step of the decoder

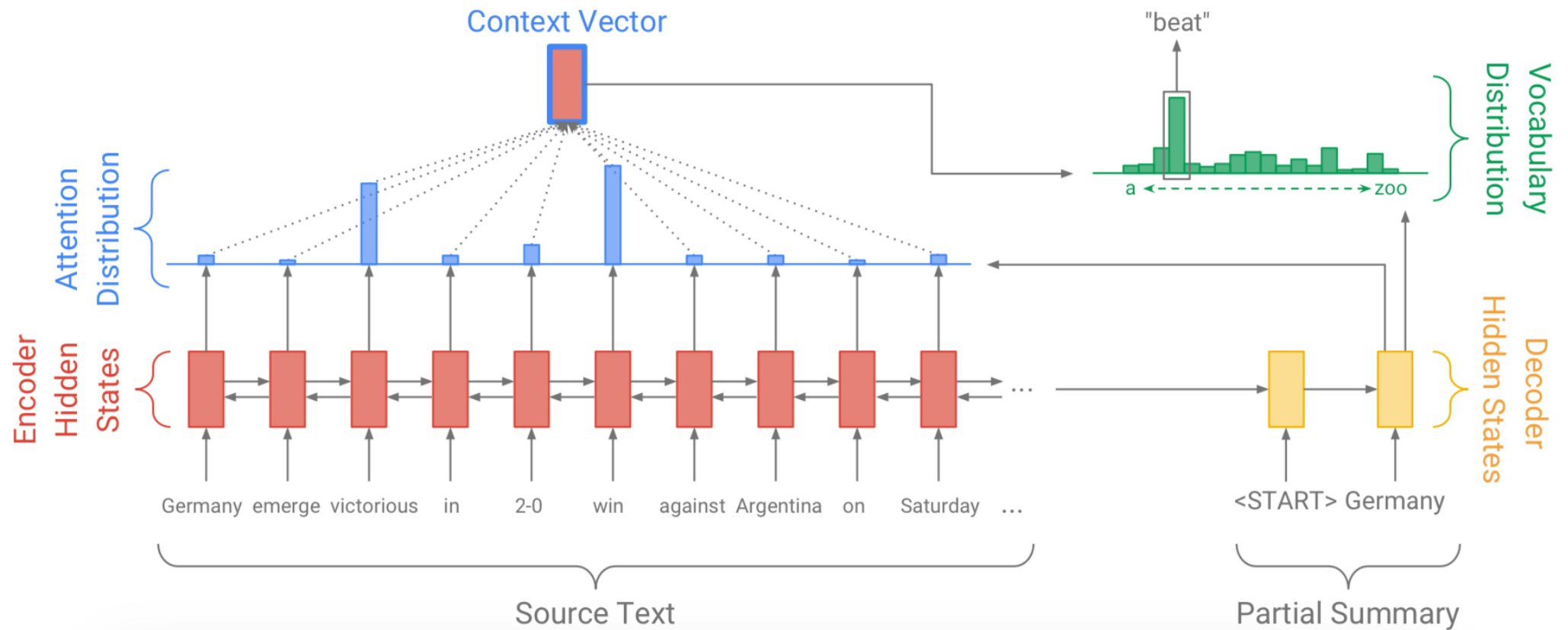
- Normalize them with a softmax
- α_{ij} : proportional relevance of each encoder hidden state j to the prior hidden decoder state h_{i-1}^d

$$\begin{aligned}\alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) \quad \forall j \in e) \\ &= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}\end{aligned}$$

Given the distribution in α , a fixed-length context vector for the current decoder state is a weighted average over all the encoder hidden states

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

Attentional Encoder-Decoder for Abstractive Summarization





Text Embeddings – rNNs – LSTM – seq2seq

(end of lecture)

ONE LOVE. ONE FUTURE.